Tugas Besar
IF3170 - Inteligensi Artifisial
Local Search



Dipersiapkan oleh

Benedict Darrel Setiawan (13524057)

Steven Tan (13524060)

Faris Wirakusuma Triawan (13524130)

Deskripsi Persoalan

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti **memaksimalkan** fungsi objektif.

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	“RusdiPackage”	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar **(seluruh bagian paket di luar)**

- Koordinat berupa bilangan bulat yang merepresentasikan titik belakang-kiri-bawah paket.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*
- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. *Value* pada *state* tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**

  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.

- **Kardus tidak bisa melayang dan menembus dinding peti kemas**

  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

$$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i, y_i \leq y < y_i + d_i\})$$

Sebuah titik di alas memiliki *support* apabila:

1. $Z_{i=0}$, artinya di alas truk, atau
2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*

- **Truk tidak boleh mengangkut melebihi kapasitasnya**

Pembahasan

1. Representasi Dan Evaluasi State

1.1. Data Struct

1.1.1. Penjelasan dan Implementasi

Untuk struktur data yang digunakan untuk semua algoritma local search dibagi menjadi beberapa bentuk class dalam bahasa pemrograman python

1. Package

   Representasi kumpulan atribut statis milik package yang bersifat statis dan tidak akan berubah dalam perjalanan algoritma local search.

```
class Package:
    def __init__(self,
```

```
                id : str,
                dimensions : dict[str, int],
                value : int,
                weight : int,
                isFragile : bool,
                eta : int) :
        self.id = id
        self.dimensions = dimensions
        self.value = value
        self.weight = weight
        self.isFragile = isFragile
        self.eta = eta
```

2. PackageState

   Representasi state package yang mengandung sebuah object Package serta variabel yang akan dapat diubah dalam perjalanan local search

```
class PackageState:
    def __init__(self,
                package : Package,
                coordinates : dict[str, int],
                orientation : dict[str, int]) :
        self.package = Package(**package) if
isinstance(package, dict) else package
        self.coordinates = coordinates
        self.orientation = orientation
```

3. Truck

   Representasi kumpulan atribut statis milik truck yang bersifat statis dan tidak akan berubah dalam perjalanan algoritma local search.

```
class Truck:
    def __init__(self,
                 dimensions : dict[str, int],
                 maxCapacity : int) :
        self.dimensions = dimensions
        self.maxCapacity = maxCapacity
```

4. TruckState

   Representasi state utama yang akan digunakan algoritma local search. Setiap TruckState mengandung objek Truck dan sebuah list yang berisi object PackageState yang merepresentasikan lokasi dan orientasi sebuah package di dalam truk. Jika PackageState memiliki koordinat yang berada di luar dimensi yang didefinisikan oleh objek Truck, maka ia dianggap berada diluar truk.

```
class TruckState:
    def __init__(self,
                 truck : Truck,
                 packages : list[PackageState]) :
        self.truck = Truck(**truck) if
isinstance(truck, dict) else truck
        self.packages = [(PackageState(**package)
if isinstance(package, dict) else package) for
package in packages]
```

1.2. Objective Function

1.2.1. Penjelasan

Objective function yang digunakan bertujuan untuk mengarahkan algoritma-algoritma local search untuk menuju state yang valid dan merupakan global optimum, serta mencegah dirinya untuk berhenti di state yang sebenarnya bukan global optimum.

Nilai objective akan dihitung berdasarkan nilai value dari setiap package yang akan dikurangi nilai penalti jika terjadi pelanggaran konstrain. Dengan ini nilai objektif tidak akan melakukan overestimasi dalam perhitungan nilai asli truck.

Berikut merupakan cara objective function mengatasi setiap constraint yang didefinisikan

1. Package tidak boleh melayang

   Konstrain ini diatasi dengan cara membuat value paket yang melayang menjadi 0 dan memberi penalti berdasarkan jarak terdekat paket dengan sebuah *support*. *Support* yang dimaksud dapat berupa lantai dari truk (koordinat z = 0) atau sebuah paket lain yang tidak fragile.

2. Package tidak boleh menembus dinding truk atau package lain

   Konstrain ini diatasi dengan memberi penalti berdasarkan volume paket yang menembus dinding truk atau package lain dikalikan dengan nilai paket per satuan volume

3. Package yang fragile tidak boleh menjadi *support* untuk package lain

   Konstrain ini diatasi dengan memberi penalti berdasarkan area paket fragile yang dijadikan support untuk paket lain

4. Truck tidak boleh mengangkut melebihi kapasitasnya

   Konstrain ini diatasi dengan memberi penalti berdasarkan besar berat yang melebihi kapasitas maksimal truck

Selain itu juga terdapat beberapa heuristik yang membantu algoritma untuk tidak berhenti saat mencapai state yang bukan merupakan global optimum

1. Memaksimalkan kepadatan package dalam truck

Hal ini dilakukan agar algoritma pasti memilih state yang memiliki penataan paket yang lebih padat. Implementasi dilakukan dengan cara memberi bobot value yang lebih besar kepada paket ketika permukaannya menyentuh permukaan paket lain atau permukaan truk

2. Memastikan package yang fragile diletakkan diatas

   Hal ini dilakukan agar mencegah algoritma untuk memilih state yang menaruh paket fragile di koordinat z yang rendah. Implementasi dilakukan dengan cara memberi bobot value yang lebih besar kepada paket fragile yang diletakkan di koordinat z yang tinggi

1.2.2. Implementasi

```
def objectiveFunction(state : TruckState) :
    value = 0.0
    weight = 0
    maxValPerUnit = 0
    truckTotalSurface = 0
    penalty = 0
    maxETA = 0
    for p in state.packages :
        maxETA = max(maxETA, p.getETA())

    for p in state.packages :
        if p.isInTruck(state.truck.dimensions) :
            weight += p.getWeight()
            maxValPerUnit = max(maxValPerUnit,
p.getValuePerUnit())
            nearestSupport = p.coordinates['z']
            intersecting = 0
            overflow =
p.overflowVolume(state.truck.dimensions)
            supportingArea = 0
```

```
            surfaceContact = contactAreaTruck(p,
state.truck.dimensions)
            truckTotalSurface += contactAreaTruck(p,
state.truck.dimensions)
            amount = 0
            late_amount = 0
            for q in state.packages :
                if p.getID() != q.getID() and
q.isInTruck(state.truck.dimensions) :
                    intersecting += intersectingArea(p,
q)*0.25
                    if canBeSupport(p, q,
state.truck.dimensions) :
                        nearestSupport =
min(nearestSupport, nearestSupportDistance(p, q))
                    if p.isFragile() :
                        supportingArea +=
supportedArea(q, p)
                    surfaceContact += contactArea(p, q)
                    if isBehindOrBelow(p, q) :
                        amount += 1
                        if p.getETA() < q.getETA() :
                            late_amount += 1

            max_p_height = state.truck.dimensions['h']
- p.package.dimensions[p.orientation['z']]
            fragilePlacement = (max_p_height -
p.coordinates['z'])/max_p_height if p.isFragile() else
0
            valid = 0 if nearestSupport or
supportingArea else 1
            lateness = (1 - late_amount/amount) if
amount > 0 else 1
            multiplier = (0.9 +
0.05*(surfaceContact)/p.getSurfaceArea() + 0.05*(1 -
fragilePlacement))
            value += p.getValue()*valid*multiplier
            penalty += (overflow*2 + nearestSupport +
intersecting + supportingArea)*p.getValuePerUnit()
```

```
    value *= (0.9975 +
0.0025*float(truckTotalSurface)/state.truck.getSurfaceA
rea())
    penalty += max(0, (weight -
state.truck.maxCapacity)*maxValPerUnit*10)
    return value - penalty
```

2. Implementasi Algoritma Local Search

2.1. Base Hill Climbing Class

2.1.1. Penjelasan

Base class hill climbing berfungsi sebagai template class sebagai helper dari semua algoritma pencarian hill climbing , keuntungannya semua algoritma Hill climbing hanya perlu mengaplikasikan/mengoverwrite base function dari class ini, serta melakukan perhitungan heuristik dari masing masing turunan hill climbing.dikarenakan fungsi rotate,pencarian neighbor sudah diwakilkan oleh fungsi parent ini

2.1.2. Implementasi

Base Class:

```
class HillClimbing:
   def __init__(self, initial_state: TruckState,
max_iterations: int = 1000, step_size: int = 2,
random_init: bool = True):
```

Melakukan Rotasi di suatu Iterasi:

```
def rotate_90(self, orientation: dict[str, str], axis:
str) -> dict[str, str]:
        new_ori = dict(orientation)
        if axis == "x":
            new_ori["y"], new_ori["z"] = new_ori["z"],
new_ori["y"]
        elif axis == "y":
            new_ori["x"], new_ori["z"] = new_ori["z"],
new_ori["x"]
        elif axis == "z":
            new_ori["x"], new_ori["y"] = new_ori["y"],
new_ori["x"]
        return new_ori
```

Pencarian Neighbor untuk HC Steepest Ascent , HC Sideways ,HC Random Restart:

```
    # buat HC steppest.sideway etc yang ngambil semua
sample neighbor
    def get_all_neighbors(self, state: TruckState) ->
list[TruckState]:
        all_neighbors: list[TruckState] = []
        n_pkgs = len(state.packages)
        if n_pkgs == 0:
            return all_neighbors

        _clone = getattr(state, "clone", lambda:
copy.deepcopy(state))
        truck_dims = state.truck.dimensions
        w_max = max(0, truck_dims.get("w", 10) - 1)
        l_max = max(0, truck_dims.get("l", 10) - 1)
        h_max = max(0, truck_dims.get("h", 10) - 1)
        for i in range(n_pkgs):
            for j in range(i + 1, n_pkgs):
```

```
                nbr = _clone()
                p1 = nbr.packages[i]
                p2 = nbr.packages[j]
                p1.coordinates, p2.coordinates =
dict(p2.coordinates), dict(p1.coordinates)
                p1.orientation, p2.orientation =
dict(p2.orientation), dict(p1.orientation)
                all_neighbors.append(nbr)
        for i in range(n_pkgs):
            for axis in ["x", "y", "z"]:
                nbr = _clone()
                nbr.packages[i].orientation =
self.rotate_90(nbr.packages[i].orientation, axis)
                all_neighbors.append(nbr)
        step = max(1, self.step_size)
        space_matrix = state.getSpaceMatrix()

        for i in range(n_pkgs):
            pkg = state.packages[i]
            if pkg.isInTruck(state.truck.dimensions):
                for x_start in range(0, w_max -
pkg.package.dimensions[pkg.orientation['x']] + 1,
step):
                    for y_start in range(0, l_max -
pkg.package.dimensions[pkg.orientation['y']] + 1,
step):
                        for z_start in range(0, h_max -
pkg.package.dimensions[pkg.orientation['z']] + 1,
step):
                            if z_start == 0 or
(not(space_matrix[x_start][y_start][z_start]) and
space_matrix[x_start][y_start][z_start - 1] == 1):
                                nbr = _clone()

nbr.packages[i].coordinates = {"x": x_start, "y":
y_start, "z": z_start}

all_neighbors.append(nbr)
```

```
                nbr = _clone()
                nbr.packages[i].coordinates = {"x": -1,
"y": -1, "z": -1}
                all_neighbors.append(nbr)

            else:
                for x_start in range(0, w_max -
pkg.package.dimensions[pkg.orientation['x']] + 1,
step):
                    for y_start in range(0, l_max -
pkg.package.dimensions[pkg.orientation['y']] + 1,
step):
                        for z_start in range(0, h_max -
pkg.package.dimensions[pkg.orientation['z']] + 1,
step):
                            if z_start == 0 or
(not(space_matrix[x_start][y_start][z_start]) and
space_matrix[x_start][y_start][z_start - 1] == 1):
                                nbr = _clone()

nbr.packages[i].coordinates = {"x": x_start, "y":
y_start, "z": z_start}

all_neighbors.append(nbr)

        return all_neighbors
```

Pencarian Neighbor untuk HC Simulated Annealing, dan HC Stoctastic:

```
# buat HC SA,stocastic
    def get_neighbor(self, state: TruckState) ->
TruckState:
        neighbor = state.clone() if hasattr(state,
'clone') else copy.deepcopy(state)
        n_pkgs = len(neighbor.packages)
        if n_pkgs == 0:
```

```
            return neighbor

        # Pilih salah satu dari 3 aksi yang
diperbolehkan
        actions = ["move", "rotate"]
        if n_pkgs >= 2:
            actions.append("swap")

        action = random.choice(actions)
        truck_dims = neighbor.truck.dimensions

        if action == "swap":
            # 1. Menukar dua paket, baik di dalam maupun
di luar truk
            i, j = random.sample(range(n_pkgs), 2)
            p1 = neighbor.packages[i]
            p2 = neighbor.packages[j]
            p1.coordinates, p2.coordinates =
copy.deepcopy(p2.coordinates),
copy.deepcopy(p1.coordinates)
            p1.orientation, p2.orientation =
copy.deepcopy(p2.orientation),
copy.deepcopy(p1.orientation)

        elif action == "move":
            # 2. Memindahkan sebuah paket ke suatu
koordinat integer yang berbeda (di dalam atau di luar
truk)
            pkg = random.choice(neighbor.packages)
            move_mode = random.choice(["shift",
"random_inside", "toggle_outside"])

            if move_mode == "shift" and
pkg.coordinates["x"] >= 0:
                # Pergeseran lokal di dalam truk
                step = self.step_size
                pkg.coordinates["x"] = max(0,
min(truck_dims.get("w", 10) - 1, pkg.coordinates["x"] +
random.randint(-step, step)))
```

```
                pkg.coordinates["y"] = max(0,
min(truck_dims.get("l", 10) - 1, pkg.coordinates["y"] +
random.randint(-step, step)))
                pkg.coordinates["z"] = max(0,
min(truck_dims.get("h", 10) - 1, pkg.coordinates["z"] +
random.randint(-step, step)))
            elif move_mode == "random_inside" or
(move_mode == "shift" and pkg.coordinates["x"] < 0):
                # Pindahkan ke koordinat integer acak di
dalam truk
                pkg.coordinates = {
                    "x": random.randint(0, max(0,
truck_dims.get("w", 10) - 1)),
                    "y": random.randint(0, max(0,
truck_dims.get("l", 10) - 1)),
                    "z": random.randint(0, max(0,
truck_dims.get("h", 10) - 1))
                }
            elif move_mode == "toggle_outside":
                if pkg.coordinates["x"] >= 0:
                    # Keluarkan paket ke luar peti kemas
truk
                    pkg.coordinates = {"x": -1, "y": -1,
"z": -1}
                else:
                    # Masukkan paket dari luar ke dalam
truk
                    pkg.coordinates = {
                        "x": random.randint(0, max(0,
truck_dims.get("w", 10) - 1)),
                        "y": random.randint(0, max(0,
truck_dims.get("l", 10) - 1)),
                        "z": random.randint(0, max(0,
truck_dims.get("h", 10) - 1))
                    }

        elif action == "rotate":
            # 3. Merotasi paket 90° terhadap sumbu x, y,
atau z
```

```
            pkg = random.choice(neighbor.packages)
            axis = random.choice(["x", "y", "z"])
            pkg.orientation =
self.rotate_90(pkg.orientation, axis)

        return neighbor
```

2.2. Steepest Ascent Hill Climbing

2.2.1. Penjelasan

Steepest Ascent Hill Climbing akan mengeksplorasi setiap neighbor state dari current state dan memilih sebuah neighbor state menjadi current state yang baru jika $h\,neighbor > h\,current$ terpenuhi.

2.2.2. Implementasi

```
class SteepestHillClimbing(HillClimbing):
    def __init__(self, initial_state: TruckState,
max_iterations: int = 1000, step_size: int = 2,
random_init: bool = True):
        super().__init__(initial_state=initial_state,
max_iterations=max_iterations, step_size=step_size,
random_init=random_init)

    def optimize(self) -> TruckState:
            start_time = time.time()
            current_cost =
objectiveFunction(self.current_state)
            best_cost = current_cost

            self.history = [{
                "iteration": 0,
                "current_cost": current_cost,
                "best_cost": best_cost,
                "accepted": True
```

```
            }]
            self.accepted_moves = 0
            self.total_iterations = 0

            for i in range(1, self.max_iterations + 1):
                self.total_iterations = i
                neighbors =
self.get_all_neighbors(self.current_state)
                best_neighbor = -1
                best_neighbor_val = current_cost
                for k in range(len(neighbors)) :
                    neighbor_cost =
objectiveFunction(neighbors[k])
                    if neighbor_cost >
best_neighbor_val :
                        best_neighbor = k
                        best_neighbor_val =
neighbor_cost

                accepted = best_neighbor != -1
                if accepted:
                    self.current_state =
neighbors[best_neighbor]
                    current_cost =
objectiveFunction(self.current_state)
                    self.accepted_moves += 1

                    if current_cost > best_cost:
                        self.best_state =
copy.deepcopy(self.current_state)
                        best_cost = current_cost
                else:
                     break

                self.history.append({
                    "iteration": i,
                    "current_cost": current_cost,
                    "best_cost": best_cost,
                    "accepted": accepted
```

```
                })



            self.execution_time = time.time() -
start_time
            return self.best_state
```

2.3. Sideways Move Hill Climbing

2.3.1. Penjelasan

Sideways Move Hill Climbing merupakan variasi Hill Climbing yang memperbolehkan perpindahan ke *neighbor* dengan nilai *objective function* yang sama dengan *current state*. Hal ini memungkinkan algoritma melewati daerah datar (*plateau*) yang dapat menghentikan Steepest Ascent Hill Climbing. Namun, untuk menghindari perpindahan berulang tanpa peningkatan, jumlah *sideways moves* berturut-turut dibatasi oleh parameter `max_sideways_moves`.

Pada setiap iterasi, algoritma memprioritaskan *neighbor* dengan nilai lebih tinggi. Jika tidak ditemukan, algoritma dapat menerima *neighbor* dengan nilai sama selama batas *sideways moves* belum tercapai. Penghitung *sideways moves* diatur kembali menjadi nol ketika ditemukan peningkatan. Pencarian berhenti apabila tidak ada perpindahan yang dapat diterima, batas *sideways moves* tercapai, atau jumlah iterasi maksimum telah digunakan.

2.3.2. Implementasi

```
def optimize(self) -> TruckState:
    start_time = time.time()
    current_cost =
objectiveFunction(self.current_state)
    best_cost = current_cost
```

```
    sideways_moves = 0

    self.history = [{
        "iteration": 0,
        "current_cost": current_cost,
        "best_cost": best_cost,
        "accepted": True
    }]
    self.accepted_moves = 0
    self.total_iterations = 0

    for i in range(1, self.max_iterations + 1):
        self.total_iterations = i
        neighbors =
self.get_all_neighbors(self.current_state)
        best_neighbor = -1
        best_neighbor_val = current_cost

        for j in range(len(neighbors)):
            neighbor_cost =
objectiveFunction(neighbors[j])

            if neighbor_cost > best_neighbor_val:
                best_neighbor = j
                best_neighbor_val = neighbor_cost
            elif (neighbor_cost == current_cost
                    and best_neighbor == -1
                    and sideways_moves <
self.max_sideways_moves):
                best_neighbor = j

        accepted = best_neighbor != -1

        if accepted:
            is_sideways = best_neighbor_val ==
current_cost

            if is_sideways:
                sideways_moves += 1
```

```
            else:
                sideways_moves = 0

            self.current_state =
neighbors[best_neighbor]
            current_cost = best_neighbor_val
            self.accepted_moves += 1

            if current_cost > best_cost:
                self.best_state =
copy.deepcopy(self.current_state)
                best_cost = current_cost
        else:
            break

        self.history.append({
            "iteration": i,
            "current_cost": current_cost,
            "best_cost": best_cost,
            "accepted": accepted
        })

        if is_sideways and sideways_moves >=
self.max_sideways_moves:
            break

    self.execution_time = time.time() - start_time
    return self.best_state
```

2.4. Stochastic Hill Climbing

2.4.1. Penjelasan

Stochastic Hill Climbing menggunakan pendekatan heuristic dengan pendekatan peluang dengan *greedy* jika *P neighbor > P current* maka successornya adalah neighbor tersebut Pada T > 0: Menerima langkah perbaikan (delta > 0) secara probabilistik menggunakan fungsi logistic: P = 1 / (1 + exp(-delta/T)).

2.4.2. Implementasi

Stochastic Hill Climbing menggunakan kelas turunan dari class *HillClimbing* :

```
class StochasticHillClimbing(HillClimbing):
    """
    Stochastic Hill Climbing.
    - Pada T > 0: Menerima langkah perbaikan (delta > 0) secara probabilistik menggunakan fungsi logistic: P = 1 / (1 + exp(-delta/T)).
    - Pada T <= 0: Berperilaku murni GREEDY (hanya menerima jika delta > 0 dengan kepastian 100%).
    - Langkah delta <= 0 selalu ditolak.
    """
    def __init__(self, initial_state: TruckState, max_iterations: int = 1000, step_size: int = 2, temperature: float = 1.0, random_init: bool = True):
        super().__init__(initial_state=initial_state, max_iterations=max_iterations, step_size=step_size, random_init=random_init)
        self.temperature = float(temperature)
```

Pencarian Heuristic&Successor State:

```
def should_accept(self, current_cost: float, neighbor_cost: float) -> bool:
        # Penolakan mutlak untuk state yang lebih buruk atau sama
        if neighbor_cost <= current_cost:
            return False

        delta = neighbor_cost - current_cost

        # Handler saat suhu 0 atau mendekati 0:
```

```
Berperilaku murni GREEDY (100% terima jika delta > 0)
        if self.temperature <= 0.0:
            return True

        scaled_delta = delta / self.temperature

        # Hindari math overflow / underflow pada exp
        if scaled_delta > 700:
            return True
        elif scaled_delta < -700:
            return False

        probability = 1.0 / (1.0 +
math.exp(-scaled_delta))
        return random.random() < probability
```

2.5. Random Restart Hill Climbing

2.5.1. Penjelasan

Random Restart menggunakan pendekatan Sideways Move Hill Climbing + Random restart ketika *stuck* pada iterasi tertentu agar tidak lock di sebuah state,sehingga pencarian neighbor dan pendekatan heuristik pada random restart sama dengan pendekatan sideway yaitu $h\,neighbor \geq h\,current$ sehingga successor yang diambil juga sama yaitu yang memenuhi $h\,neighbor \geq h\,current$

2.5.2. Implementasi

Simulated Anneiling menggunakan kelas turunan dari class *HillClimbing* :

```
class RandomRestartHillClimbing(HillClimbing):
    def __init__(self, initial_state: TruckState,
max_iterations: int = 1000, step_size: int = 2,
random_init: bool = True, max_restarts: int = 10):
        super().__init__(initial_state=initial_state,
max_iterations=max_iterations, step_size=step_size,
random_init=random_init)
        self.max_restarts = max_restarts
```

Pencarian Heuristic&Successor State:

```
    def optimize(self) -> TruckState:
        start_time = time.time()
        best_state = self.current_state
        best_cost = objectiveFunction(best_state)
        self.accepted_moves = 0
        self.history = []
        self.run_iterations: list[int] = []
        self.total_iterations = 0

        for restart in range(self.max_restarts + 1):
            if restart > 0:
                self.current_state =
self.initial_state.generate_random_state()
            current_cost =
objectiveFunction(self.current_state)
            run_steps = 0

            if current_cost > best_cost:
                best_state =
copy.deepcopy(self.current_state)
                best_cost = current_cost
```

```
            self.history.append({
                "iteration": self.total_iterations,
                "current_cost": current_cost
            })

            for _ in range(self.max_iterations):
                self.total_iterations += 1
                run_steps += 1

                neighbors =
self.get_all_neighbors(self.current_state)
                best_neighbor = -1
                best_neighbor_val = current_cost

                for i in range(len(neighbors)):
                    neighbor_cost =
objectiveFunction(neighbors[i])

                    if neighbor_cost >
best_neighbor_val:
                        best_neighbor = i
                        best_neighbor_val =
neighbor_cost

                if best_neighbor == -1:
                    break  # No better neighbor found,
exit the loop

                self.current_state =
neighbors[best_neighbor]
                current_cost = best_neighbor_val
                self.accepted_moves += 1

                if current_cost > best_cost:
                    best_state =
copy.deepcopy(self.current_state)
                    best_cost = current_cost

                self.history.append({
```

```
                    "iteration": self.total_iterations,
                    "current_cost": current_cost
                })

            self.run_iterations.append(run_steps)

        self.execution_time = time.time() - start_time
        self.best_state = best_state
        return best_state
```

2.6. Simulated Annealing

2.6.1. Penjelasan

Simulated Annealing menggunakan pendekatan heuristik dengan formula:

$$P = e^{-(\Delta E/T)}$$

ΔE didapatkan dari perhitungan heuristik stochastic hill climbing,variabel T atau suhu di atur secara manual semakin tinggi suhu (T > 0) maka akan semakin random,sebaliknya jika suhu semakin rendah (T <= 0) semakin mendekati algoritma *greedy.*

Pengaktivasian initial state dilakukan dengan sampel random,namun untuk mencari ketetanggan sama seperti stocastic tidak mengaktifkan *neighbors* seluruhnya namun hanya memanggil sampelnya saja

2.6.2. Implementasi

Simulated Anneiling menggunakan kelas turunan dari class *StochasticHillClimbing* :

```
class SimulatedAnnealing(StochasticHillClimbing):
    def __init__(self,
                 initial_state: TruckState,
                 initial_temp: float = 1000.0,
                 cooling_rate: float = 0.995,
                 min_temp: float = 0.001,
                 max_iterations: int = 1000,
                 steps_per_temp: int = 1,
                 cooling_schedule: Union[str,
Callable[[float, int], float]] = "geometric",
                 step_size: int = 2,
                 random_init: bool = True):
        super().__init__(initial_state=initial_state,
max_iterations=max_iterations, step_size=step_size,
random_init=random_init)
        self.initial_temp = float(initial_temp)
        self.temp = float(initial_temp)
        self.cooling_rate = float(cooling_rate)
        self.min_temp = float(min_temp)
        self.steps_per_temp = max(1,
int(steps_per_temp))
        self.cooling_schedule = cooling_schedule

        # Metrics tracking
        self.accepted_better_moves: int = 0
        self.accepted_worse_moves: int = 0
        self.rejected_moves: int = 0
        self.temperatures: list[float] = []
        self.costs: list[float] = []
```

Pencarian Heuristic:

```
def acceptance_probability(self, delta: float, temp:
float) -> float:
        """
```

```
        Menghitung probabilitas penerimaan Boltzmann P =
exp(delta / T).
        - delta > 0  : Selalu return 1.0 (langkah
perbaikan).
        - temp <= 0  : Mode GREEDY -> return 1.0 jika
delta > 0, return 0.0 jika delta <= 0.
        - temp -> inf: Mode RANDOM WALK -> return 1.0
untuk semua langkah (karena delta/T -> 0, exp(0) = 1).
        - delta <= 0 : return exp(delta / temp) dalam
rentang (0.0, 1.0].
        """
        if delta > 0:
            return 1.0

        if temp <= 0.0:
            # Pada suhu 0 (nol mutlak), tidak ada
toleransi langkah buruk -> Greedy
            return 0.0

        if math.isinf(temp) or temp >= 1e12:
            # Pada suhu sangat tinggi / maksimum, semua
langkah diterima -> Pure Random Walk
            return 1.0

        exponent = delta / temp
        if exponent < -700:
            return 0.0
        elif exponent > 0:
            return 1.0

        return math.exp(exponent)
```

Pengambilan successor:

```
while self.temp > self.min_temp and iteration <
self.max_iterations:
            iteration += 1
            step_counter += 1

            neighbor =
self.get_neighbor(self.current_state)
            neighbor_cost = objectiveFunction(neighbor)
            delta = neighbor_cost - current_cost

            accepted, prob =
self.should_accept(current_cost, neighbor_cost,
self.temp)

            if accepted:
                self.current_state = neighbor
                current_cost = neighbor_cost
                self.accepted_moves += 1

                if delta > 0:
                    self.accepted_better_moves += 1
                else:
                    self.accepted_worse_moves += 1

                if current_cost > best_cost:
                    self.best_state =
copy.deepcopy(self.current_state)
                    best_cost = current_cost
            else:
                self.rejected_moves += 1

def compute_next_temp(self, current_temp: float,
iteration: int) -> float:
        if callable(self.cooling_schedule):
            return max(0.0,
self.cooling_schedule(current_temp, iteration))

        schedule = str(self.cooling_schedule).lower()
```

```
        if schedule in ("geometric", "exponential"):
            return current_temp * self.cooling_rate
        elif schedule == "linear":
            return max(self.min_temp, current_temp -
self.cooling_rate)
        elif schedule == "logarithmic":
            # T_k = T_0 / (1 + cooling_rate * ln(1 + k))
            return self.initial_temp / (1.0 +
self.cooling_rate * math.log(1.0 + iteration))
        elif schedule == "cosine":
            # T_k = min_temp + 0.5 * (T_0 - min_temp) *
(1 + cos(pi * k / K_max))
            progress = min(1.0, iteration / max(1,
self.max_iterations))
            return self.min_temp + 0.5 *
(self.initial_temp - self.min_temp) * (1.0 +
math.cos(math.pi * progress))
        else:
            # Default fallback to geometric
            return current_temp * self.cooling_rate
```

2.7. Genetic Algorithm

2.7.1. Penjelasan

Genetic Algorithm (GA) merupakan algoritma *local search* berbasis populasi yang mengembangkan solusi melalui proses seleksi, *crossover*, dan mutasi. Pada persoalan pemuatan paket, setiap individu merepresentasikan kandidat konfigurasi pemuatan truk. Implementasi menggunakan representasi kromosom tidak langsung yang diterjemahkan menjadi konfigurasi fisik melalui

proses *decoding*, sehingga algoritma dapat mengeksplorasi pemilihan paket, urutan pemuatan, dan orientasi penempatan.

a. Genetic Representation

Setiap individu direpresentasikan oleh kelas `Chromosome` yang terdiri atas kumpulan `Gene` dan atribut `placement_order`. Setiap gen menyimpan `id` paket, `loaded_flag` untuk menentukan apakah paket dicoba dimuat, serta `orientation` sebagai orientasi awal yang diprioritaskan. Urutan gen menentukan urutan pemuatan paket, sedangkan `placement_order` menentukan prioritas pengurutan titik penempatan. Kedua atribut orientasi menggunakan enam permutasi sumbu koordinat.

b. Population Initialization

Populasi awal dibangkitkan secara acak melalui `populationInitialization()`. Setiap kromosom memiliki urutan identitas paket yang diacak tanpa pengulangan, nilai `loaded_flag` acak antara 0 dan 1, serta `orientation` dan `placement_order` yang dipilih dari enam permutasi sumbu. Pengacakan tersebut bertujuan menghasilkan keragaman konfigurasi awal agar pencarian dapat mengeksplorasi berbagai kemungkinan solusi.

c. Chromosome Decoding

Kromosom diterjemahkan menjadi `TruckState` menggunakan *Extreme Point Heuristics*. Setiap paket yang dipilih diproses berdasarkan urutan gennya, kemudian dicoba pada titik kandidat dan orientasi yang tersedia. Penempatan diprioritaskan apabila tidak melewati batas truk, tidak bertabrakan, dan memiliki penopang yang sesuai. Setelah penempatan, kumpulan *Extreme Points* diperbarui untuk paket berikutnya.

Apabila tidak ditemukan posisi yang memenuhi pemeriksaan tersebut, algoritma menggunakan penempatan cadangan (*fallback*) yang dapat menghasilkan konfigurasi tidak valid. Pendekatan ini memungkinkan seluruh kromosom tetap dievaluasi, sementara pelanggaran ditangani melalui penalti dan pemeriksaan kelayakan.

d. Fitness Evaluation and Feasibility

Nilai *fitness* diperoleh dengan menghitung `objectiveFunction()` terhadap hasil *decoding*. Fungsi tersebut mempertimbangkan nilai paket, kualitas penempatan, serta penalti terhadap pelanggaran. Secara terpisah, `validate_feasibility()` memeriksa batas truk, tabrakan, penopang, larangan penopang rapuh, dan kapasitas berat. Pemisahan ini memungkinkan individu tidak valid tetap mengikuti evolusi, tetapi hanya solusi *feasible* yang dapat disimpan sebagai solusi akhir terbaik.

e. Parent Selection

Pemilihan induk menggunakan *Tournament Selection* dengan ukuran turnamen bawaan tiga individu. Kandidat dipilih secara acak dari populasi, kemudian individu dengan *fitness* tertinggi dipilih sebagai induk. Metode ini memprioritaskan individu berkualitas tinggi tanpa menghilangkan unsur acak dalam proses seleksi.

f. Crossover

Operasi *crossover* menggunakan *Order Crossover* (OX) dengan probabilitas 0,8. Dua titik potong dipilih secara acak, kemudian segmen gen dari induk pertama disalin ke anak. Posisi tersisa diisi menggunakan gen induk kedua sesuai urutannya tanpa mengulang identitas paket. Metode ini mempertahankan permutasi paket yang valid sekaligus menggabungkan karakteristik kedua induk.

g. Mutation

Mutasi dilakukan dengan probabilitas 0,20 untuk salah satu dari tiga operasi: menukar posisi dua gen, membalik keputusan pemuatan, atau mengubah orientasi paket. Selain itu, `placement_order` dapat dimutasi secara independen dengan probabilitas 0,05. Kedua mekanisme tersebut bertujuan mempertahankan keragaman populasi dan mengeksplorasi konfigurasi baru.

h. Replacement and Elitism

Pembentukan generasi berikutnya menggunakan *generational replacement* dengan *elitism* sekitar 2%, yaitu mempertahankan sejumlah individu dengan *fitness* tertinggi dari generasi sebelumnya. Sisa populasi dihasilkan melalui seleksi, *crossover*, dan mutasi. Selain itu, solusi *feasible* terbaik disimpan dalam arsip terpisah sepanjang proses evolusi untuk memastikan solusi valid terbaik yang pernah ditemukan dapat dikembalikan sebagai hasil akhir.

2.7.2. Implementasi

a. Genetic Representation

```
class AxisPermutation(Enum):
    XYZ = (0, 1, 2)
    XZY = (0, 2, 1)
    YZX = (1, 2, 0)
    YXZ = (1, 0, 2)
    ZXY = (2, 0, 1)
    ZYX = (2, 1, 0)

class Gene:
    def __init__(self,
                 id : str,
                 loaded_flag : int,
                 orientation : AxisPermutation)
-> None:
```

```
        self.id = id
        self.loaded_flag = loaded_flag
        self.orientation = orientation

class Chromosome:
    def __init__(self,
                 genes : list[Gene],
                 placement_order :
AxisPermutation) -> None:
        self.genes = genes
        self.placement_order = placement_order
```

b. Population Initialization

```
def populationInitialization(n : int, truckState
: TruckState,
                             rng: random.Random |
None = None) -> list[Chromosome]:
    rng = rng or random
    packages = [packageState.package for
packageState in truckState.packages]
    package_amount = len(packages)
    packages_id = [package.id for package in
packages]

    population = []
    for i in range(n):
        genes = []
        available_packages =
copy.deepcopy(packages_id)
        for j in range(package_amount):
            id = rng.choice(available_packages)
            available_packages.remove(id)
            loaded_flag = rng.randint(0, 1)
            orientation =
rng.choice(list(AxisPermutation))
            gene = Gene(id, loaded_flag,
orientation)
```

```
            genes.append(gene)
        placementOrder =
rng.choice(list(AxisPermutation))

        chromosome = Chromosome(genes,
placementOrder)
        population.append(chromosome)

    return population
```

c. Chromosome Decoding

Placement Algorithm :

```
def placement_algorithm(placement_order :
AxisPermutation, extreme_points : set[tuple[int,
int, int]],
                        cur_package_data :
Package, orientation : AxisPermutation,
                        occupied_space :
list[OccupiedPackage], truck : Truck) ->
tuple[AABB, AxisPermutation]:

    sort_axes = placement_order.value[::-1]
    # check all possible placement points
    sorted_extreme_points =
sorted(extreme_points,

key=itemgetter(*sort_axes))

    list_orientation = list(AxisPermutation)
    start_i_orientation =
list_orientation.index(orientation)
    orientation_order =
list_orientation[start_i_orientation:] +
list_orientation[:start_i_orientation]

    placed = False
```

```
    for cur_extreme_point in
sorted_extreme_points:
        # try all orientation
        for cur_orientation in orientation_order:
            cur_package_bounding_box =
AABB(cur_extreme_point,
cur_package_data.dimensions, cur_orientation)

            # reject packages extending beyond
the truck.
            if (cur_package_bounding_box.x_max >
truck.dimensions["w"]
                or cur_package_bounding_box.y_max
> truck.dimensions["l"]
                or cur_package_bounding_box.z_max
> truck.dimensions["h"]):
                continue

            # reject overlapping packages.
            if
any(detect_box_collision(cur_package_bounding_box
, occupied.box) for occupied in occupied_space):
                continue

            # check support.
            supported =
cur_package_bounding_box.z_min == 0

            if not supported:
                for occupied in occupied_space:
                    support_box = occupied.box

                    if (not occupied.is_fragile
                        and support_box.z_max ==
cur_package_bounding_box.z_min
                        and
min(support_box.x_max,
cur_package_bounding_box.x_max) >
max(support_box.x_min,
```

```
cur_package_bounding_box.x_min)
                        and
min(support_box.y_max,
cur_package_bounding_box.y_max) >
max(support_box.y_min,
cur_package_bounding_box.y_min)):
                        supported = True
                        break

            if supported:
                placed = True
                chosen_orientation =
cur_orientation
                break

        if placed:
            break

    if (not placed):
        fallback_point = sorted_extreme_points[0]
if sorted_extreme_points else (0, 0, 0)
        chosen_orientation = orientation
        cur_package_bounding_box =
AABB(fallback_point, cur_package_data.dimensions,
chosen_orientation)

    candidate_points =
candidate_point_generation(cur_package_bounding_b
ox, occupied_space)

    update_extreme_points(extreme_points,
candidate_points, occupied_space, truck,
cur_package_bounding_box)


occupied_space.append(OccupiedPackage(cur_package
_bounding_box, cur_package_data.isFragile))

    return (cur_package_bounding_box,
```

```
chosen_orientation)
```

Candidate Point Generation and Update :

```
# based on Extreme Point Heuristics
def candidate_point_generation(bounding_box :
AABB,
                               occupied_space :
list[OccupiedPackage]) -> list[tuple[int, int,
int]]:
    extreme_x = (bounding_box.x_max,
bounding_box.y_min, bounding_box.z_min)
    extreme_y = (bounding_box.x_min,
bounding_box.y_max, bounding_box.z_min)
    extreme_z = (bounding_box.x_min,
bounding_box.y_min, bounding_box.z_max)

    candidate_point_1 = (extreme_x[0],
extreme_x[1], 0)
    candidate_point_2 = (extreme_x[0], 0,
extreme_x[2])
    candidate_point_3 = (extreme_y[0],
extreme_y[1], 0)
    candidate_point_4 = (0, extreme_y[1],
extreme_y[2])
    candidate_point_5 = (extreme_z[0], 0,
extreme_z[2])
    candidate_point_6 = (0, extreme_z[1],
extreme_z[2])
    for occupied_package in occupied_space:
        box = occupied_package.box
        # x projected on xy plane
        if (extreme_x[0] >= box.x_min and
extreme_x[0] < box.x_max and
            extreme_x[1] >= box.y_min and
extreme_x[1] < box.y_max and
            extreme_x[2] > box.z_max):
            candidate_point = (extreme_x[0],
```

```
extreme_x[1], box.z_max)
            if (candidate_point[2] >
candidate_point_1[2]):
                candidate_point_1 =
candidate_point

        # x projected on xz plane
        if (extreme_x[0] >= box.x_min and
extreme_x[0] < box.x_max and
            extreme_x[2] >= box.z_min and
extreme_x[2] < box.z_max and
            extreme_x[1] > box.y_max):
            candidate_point = (extreme_x[0],
box.y_max, extreme_x[2])
            if (candidate_point[1] >
candidate_point_2[1]):
                candidate_point_2 =
candidate_point

        # y projected on xy plane
        if (extreme_y[0] >= box.x_min and
extreme_y[0] < box.x_max and
            extreme_y[1] >= box.y_min and
extreme_y[1] < box.y_max and
            extreme_y[2] > box.z_max):
            candidate_point = (extreme_y[0],
extreme_y[1], box.z_max)
            if (candidate_point[2] >
candidate_point_3[2]):
                candidate_point_3 =
candidate_point

        # y projected on yz plane
        if (extreme_y[1] >= box.y_min and
extreme_y[1] < box.y_max and
            extreme_y[2] >= box.z_min and
extreme_y[2] < box.z_max and
            extreme_y[0] > box.x_max):
            candidate_point = (box.x_max,
```

```
extreme_y[1], extreme_y[2])
            if (candidate_point[0] >
candidate_point_4[0]):
                candidate_point_4 =
candidate_point

        # z projected on xz plane
        if (extreme_z[0] >= box.x_min and
extreme_z[0] < box.x_max and
            extreme_z[2] >= box.z_min and
extreme_z[2] < box.z_max and
            extreme_z[1] > box.y_max):
            candidate_point = (extreme_z[0],
box.y_max, extreme_z[2])
            if (candidate_point[1] >
candidate_point_5[1]):
                candidate_point_5 =
candidate_point

        # z projected on yz plane
        if (extreme_z[1] >= box.y_min and
extreme_z[1] < box.y_max and
            extreme_z[2] >= box.z_min and
extreme_z[2] < box.z_max and
            extreme_z[0] > box.x_max):
            candidate_point = (box.x_max,
extreme_z[1], extreme_z[2])
            if (candidate_point[0] >
candidate_point_6[0]):
                candidate_point_6 =
candidate_point

    candidate_points = [candidate_point_1,
candidate_point_2, candidate_point_3,
                        candidate_point_4,
candidate_point_5, candidate_point_6]

    return candidate_points
```

```
def update_extreme_points(extreme_points :
set[tuple[int, int, int]],
                          candidate_points :
list[tuple[int, int, int]],
                          old_occupied_space :
list[OccupiedPackage],
                          truck : Truck,
                          new_package : AABB) ->
set[tuple[int, int, int]]:

    to_remove = {point for point in extreme_points
if is_point_in_box(point, new_package)}
    extreme_points.difference_update(to_remove)

    for candidate_point in candidate_points:
        if (candidate_point [0] < 0 or
            candidate_point[0] >=
truck.dimensions["w"] or
            candidate_point [1] < 0 or
            candidate_point[1] >=
truck.dimensions["l"] or
            candidate_point [2] < 0 or
            candidate_point[2] >=
truck.dimensions["h"]):
            continue

        if (is_point_in_box(candidate_point,
new_package)):
            continue

        if any(is_point_in_box(candidate_point,
occupied.box) for occupied in old_occupied_space):
            continue

        extreme_points.add(candidate_point)

    return extreme_points
```

Decoding :

```
def decode(truck : Truck, packages_data :
list[Package], chromosome : Chromosome) ->
TruckState:

    package_by_id = {package.id : package for
package in packages_data}

    package_states = []
    extreme_points = {(0, 0, 0)}
    occupied_space: list[OccupiedPackage] = []
    for gene in chromosome.genes:
        package_data = package_by_id[gene.id]

        loaded_flag = gene.loaded_flag
        if(loaded_flag):
            placed_package, chosen_orientation =
placement_algorithm(chromosome.placement_order,
extreme_points, package_data, gene.orientation,
occupied_space, truck)
            coordinates =
encoded_coords_to_decoded_coords((placed_package.
x_min, placed_package.y_min,
placed_package.z_min))
            orientation =
axis_permutation_to_orientation(chosen_orientatio
n)

            package_state =
PackageState(package_data, coordinates,
orientation)
        else:
            coordinates =
encoded_coords_to_decoded_coords((-1, -1, -1))
            orientation =
axis_permutation_to_orientation(gene.orientation)

            package_state =
```

```
PackageState(package_data, coordinates,
orientation)

        package_states.append(package_state)

    return TruckState(truck, package_states)
```

d. Fitness Evaluation and Feasibility

Validate Feasibility :

```
def validate_feasibility(state: TruckState) ->
bool:
    dimensions = state.truck.dimensions

    boxes = []
    total_weight = 0

    for item in state.packages:
        package = item.package
        coords = item.coordinates

        # Unloaded packages are excluded from the
physical constraints
        if (coords["x"], coords["y"],
coords["z"]) == (-1, -1, -1):
            continue

        low = tuple(coords[k] for k in ("x", "y",
"z"))
        high = tuple(
            low[i] +
package.dimensions[item.orientation[k]]
            for i, k in enumerate(("x", "y",
"z"))
        )

        # Loaded packages cannot penetrate the
```

```
truck walls
        if (high[0] > dimensions["w"] or
            high[1] > dimensions["l"] or
            high[2] > dimensions["h"]):
            return False

        boxes.append((low, high,
package.isFragile))
        total_weight += package.weight

    # Total weight of loaded packages must not
exceed maxCapacity
    if total_weight > state.truck.maxCapacity:
        return False

    for i, (low, high, _) in enumerate(boxes):
        supported = low[2] == 0

        for j, (other_low, other_high,
other_fragile) in enumerate(boxes):
            if i == j:
                continue

            xy_overlap = (
                min(high[0], other_high[0]) >
max(low[0], other_low[0])
                and min(high[1], other_high[1]) >
max(low[1], other_low[1])
            )

            if not xy_overlap:
                continue

            # Packages cannot overlap
            if min(high[2], other_high[2]) >
max(low[2], other_low[2]):
                return False

            if low[2] == other_high[2]:
```

```
                supported = True

                # Fragile package cannot support
another package
                if other_fragile:
                    return False

        # Package above the floor needs a
supporting base contact
        if not supported:
            return False

    return True
```

Evaluate Fitness :

```
def evaluate_population(population:
list[Chromosome], truck: Truck,
                        packages: list[Package],
                        archive: FeasibleRecord,
generation: int
                        ) -> tuple[list[float],
GenerationStats, FeasibleRecord]:
    fitnesses: list[float] = []
    feasible_count = 0
    for chromosome in population:
        state = decode(truck, packages,
chromosome)
        fitness = objectiveFunction(state)
        fitnesses.append(fitness)
        if validate_feasibility(state):
            feasible_count += 1
            if fitness > archive.fitness:
                archive =
FeasibleRecord(copy.deepcopy(chromosome),

copy.deepcopy(state), fitness)
```

```
    stats = GenerationStats(generation,
max(fitnesses),
                            sum(fitnesses) /
len(fitnesses), archive.fitness,
                            100 * feasible_count
/ len(population))

    return fitnesses, stats, archive
```

e. Parent Selection

```
def tournament_selection(population:
list[Chromosome], fitnesses: list[float],
                         tournament_size: int =
3,
                         rng: random.Random |
None = None) -> Chromosome:
    rng = rng or random
    winner = max((rng.randrange(len(population))
for _ in range(tournament_size)),
                 key=lambda index:
fitnesses[index])

    return population[winner]
```

f. Crossover

```
def order_crossover(parent1: Chromosome, parent2:
Chromosome,
                    rng: random.Random | None =
None) -> tuple[Chromosome, Chromosome]:
    rng = rng or random

    n = len(parent1.genes)
    start, end = sorted(rng.sample(range(n + 1),
2)) if n else (0, 0)
```

```
    children: list[Chromosome] = []
    for primary, secondary in ((parent1,
parent2), (parent2, parent1)):
        genes = [None] * n
        genes[start:end] = [copy.deepcopy(g) for
g in primary.genes[start:end]]
        used = {g.id for g in genes if g is not
None}
        fill = (list(range(end, n)) +
list(range(end)))
        donor = secondary.genes[end:] +
secondary.genes[:end]
        remaining = (g for g in donor if g.id not
in used)
        for index in fill:
            if genes[index] is None:
                genes[index] =
copy.deepcopy(next(remaining))
        placement =
rng.choice((primary.placement_order,
secondary.placement_order))
        children.append(Chromosome(genes,
placement))

    return children[0], children[1]
```

g. Mutation

```
def mutate(chromosome: Chromosome,
package_probability: float = 0.20,
           placement_probability: float = 0.05,
           rng: random.Random | None = None) ->
Chromosome:
    rng = rng or random

    result = copy.deepcopy(chromosome)
    if result.genes and rng.random() <
package_probability:
```

```
        operations = ["flip", "orientation"]
        if len(result.genes) >= 2:
            operations.append("swap")
        operation = rng.choice(operations)
        if operation == "swap":
            i, j =
rng.sample(range(len(result.genes)), 2)
            result.genes[i], result.genes[j] =
result.genes[j], result.genes[i]
        else:
            gene = rng.choice(result.genes)
            if operation == "flip":
                gene.loaded_flag = 1 -
gene.loaded_flag
            else:
                gene.orientation = rng.choice([p
for p in AxisPermutation if p !=
gene.orientation])

    if rng.random() < placement_probability:
        result.placement_order = rng.choice([p
for p in AxisPermutation
                                             if p
!= result.placement_order])

    return result
```

h. Replacement and Elitism

```
def genetic_algorithm(initial_state: TruckState,
population_size: int = 100,
                      generations: int = 200,
tournament_size: int = 3,
                      crossover_probability:
float = 0.8,

package_mutation_probability: float = 0.20,
```

```
placement_mutation_probability: float = 0.05,
                        seed: int | None = None) ->
GAResult:
    start_time = time.time()
    rng = random.Random(seed)

    packages = [item.package for item in
initial_state.packages]

    empty = Chromosome([Gene(package.id, 0,
AxisPermutation.XYZ) for package in packages],
                           AxisPermutation.XYZ)
    empty_state = decode(initial_state.truck,
packages, empty)

    record = FeasibleRecord(copy.deepcopy(empty),
copy.deepcopy(empty_state),

objectiveFunction(empty_state))

    population =
populationInitialization(population_size,
initial_state, rng)
    initial_decoded =
copy.deepcopy(decode(initial_state.truck,
packages, population[0]))

    history: list[GenerationStats] = []
    fitnesses, stats, record =
evaluate_population(
        population, initial_state.truck,
packages, record, 0)
    history.append(stats)
    for generation in range(1, generations + 1):
        elite_count = min(population_size - 1,
max(1, math.ceil(0.02 * population_size)))
        elite_indices =
sorted(range(population_size), key=lambda i:
fitnesses[i], reverse=True)[:elite_count]
```

```
        next_population =
[copy.deepcopy(population[i]) for i in
elite_indices]
        while len(next_population) <
population_size:
            parent1 =
tournament_selection(population, fitnesses,
tournament_size, rng)
            parent2 =
tournament_selection(population, fitnesses,
tournament_size, rng)

            if rng.random() <
crossover_probability:
                children =
order_crossover(parent1, parent2, rng)
            else:
                children =
copy.deepcopy(parent1), copy.deepcopy(parent2)
            for child in children:
                if len(next_population) <
population_size:

next_population.append(mutate(child,
package_mutation_probability,

placement_mutation_probability, rng))

        population = next_population
        fitnesses, stats, record =
evaluate_population(
            population, initial_state.truck,
packages, record, generation)
        history.append(stats)

    duration = time.time() - start_time

    return GAResult(initial_decoded,
```

```
record.state, record.fitness, history, duration)
```

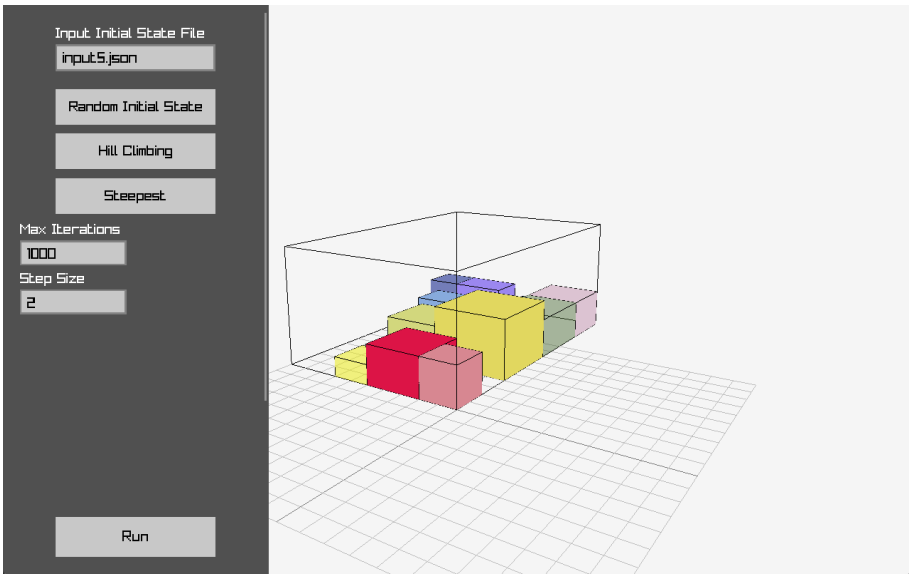
3. Hasil dan Analisis

Semua hasil percobaan menggunakan file input5.json sebagai initial state

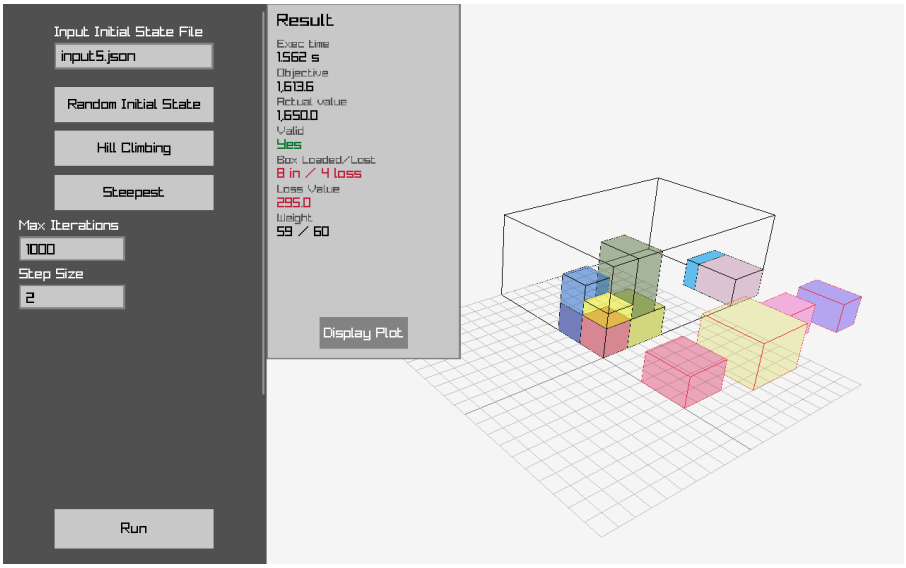
3.1. Steepest Ascent Hill Climbing

3.1.1. Hasil

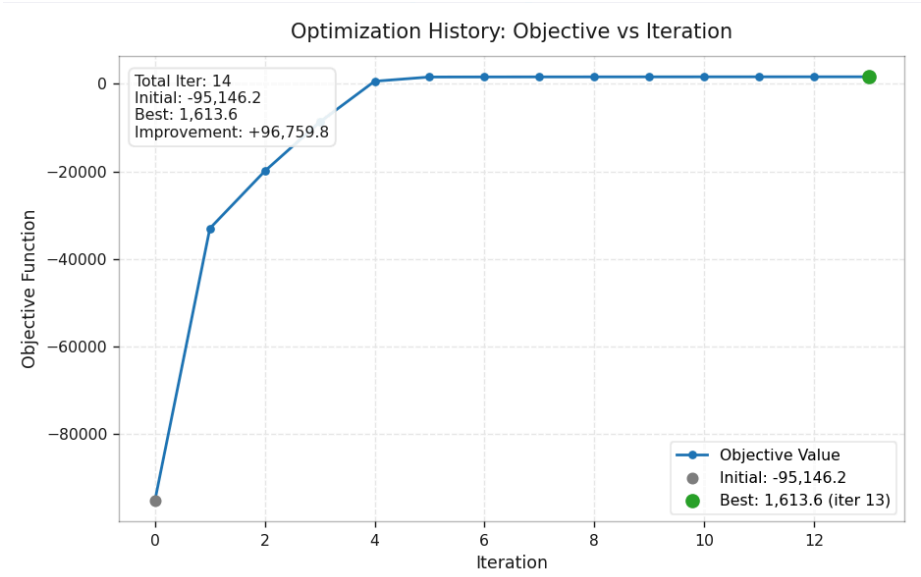
Initial :



Final :



Plot:



3.1.2. Analisis

Hasil plotting menunjukan bahwa initial state memiliki objektif ≤ -80000. Ini dikarenakan total berat paket jauh melebihi kapasitas maksimum truck, sehingga objective function memberi penalti yang sangat berat. Pada final state, algoritma Steepest

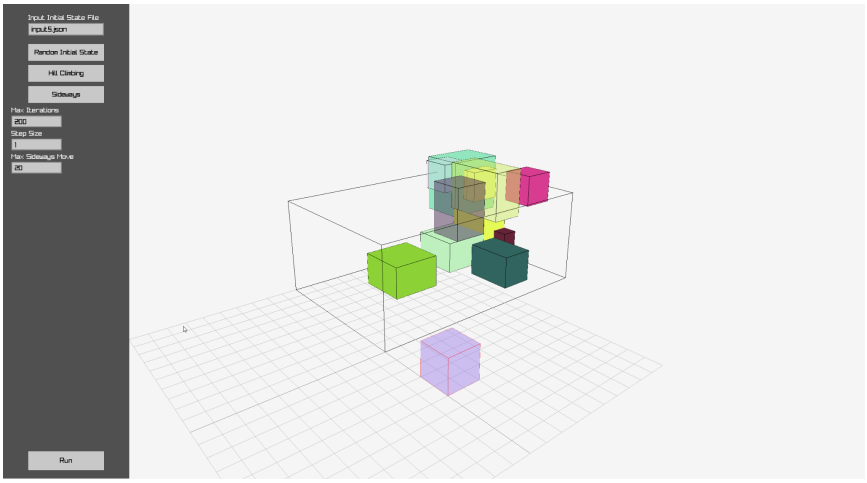
Ascent Hill Climbing berhasil mengubah initial state menjadi memiliki objective value 1613,6 dengan value asli 1650.

3.2. Sideways Move Hill Climbing

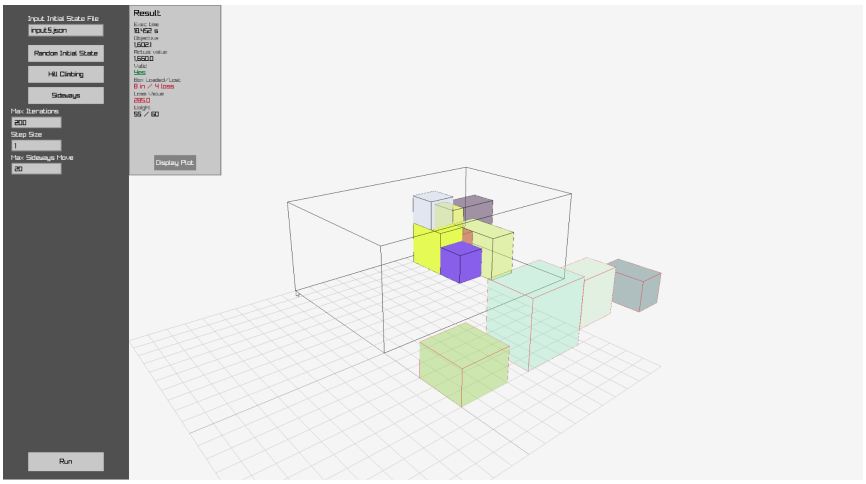
3.2.1. Hasil

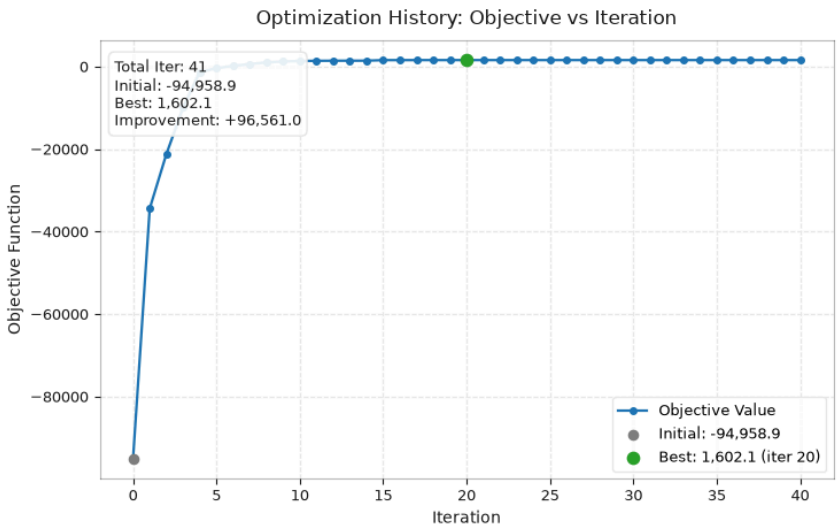
3.2.1.1. Run Pertama

Initial :



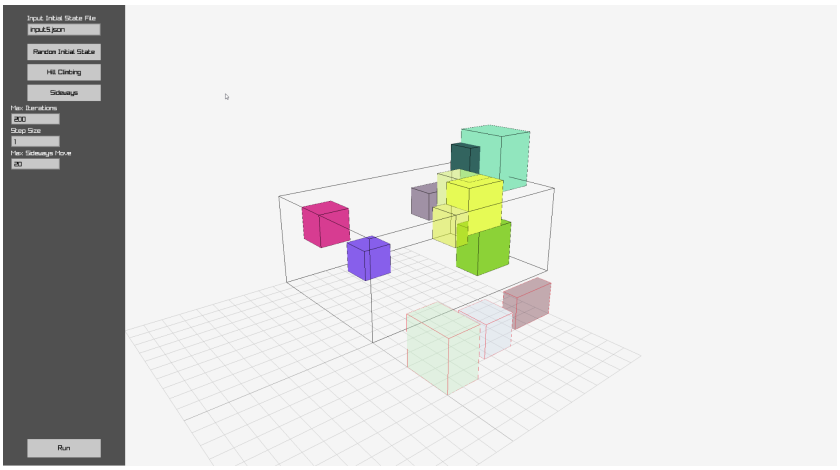
Final :



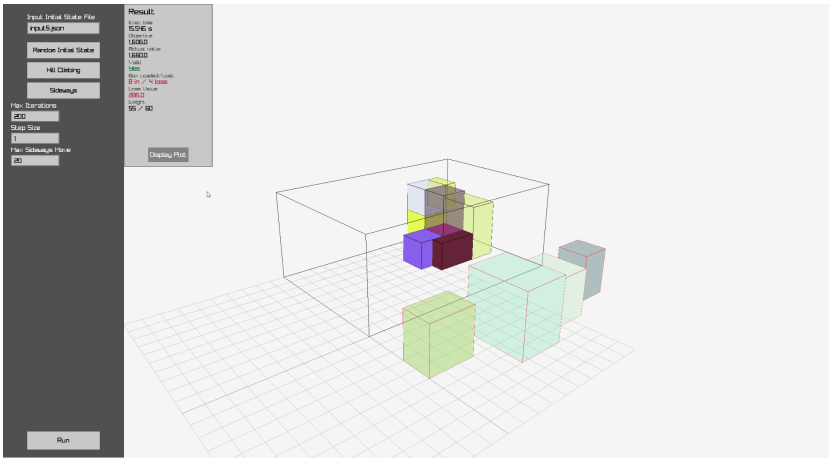


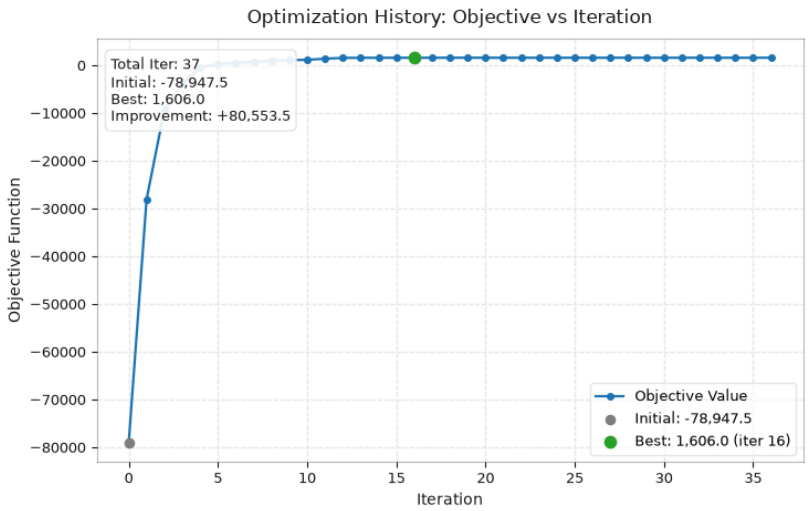
3.2.1.2. Run Kedua

Initial :



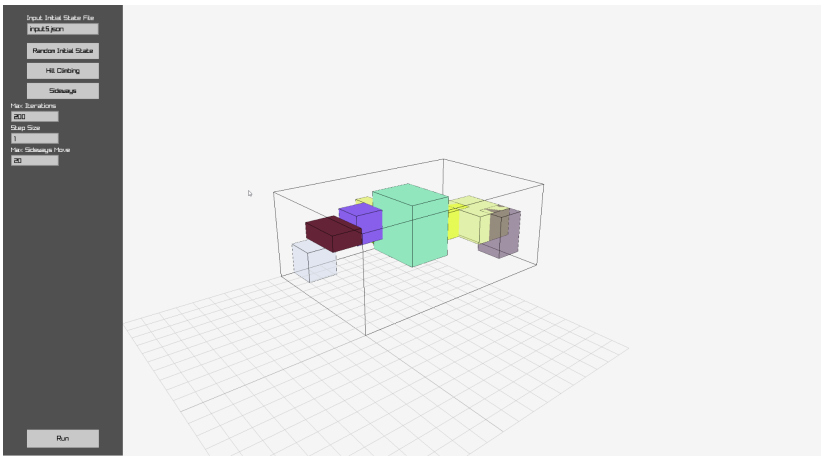
Final :



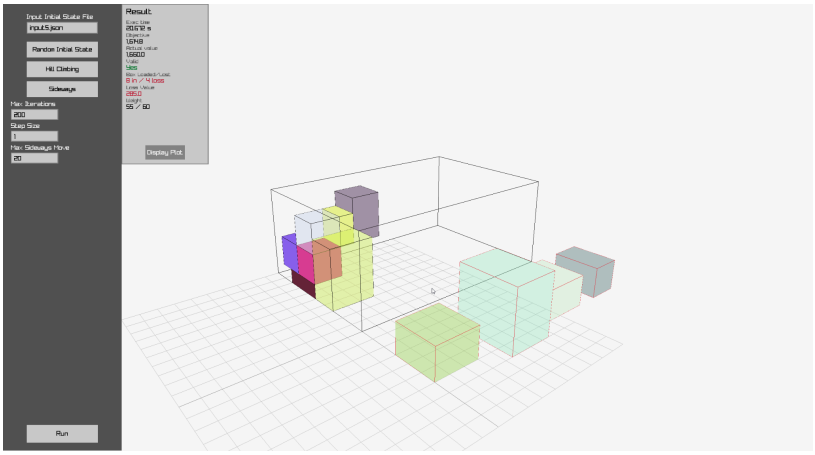


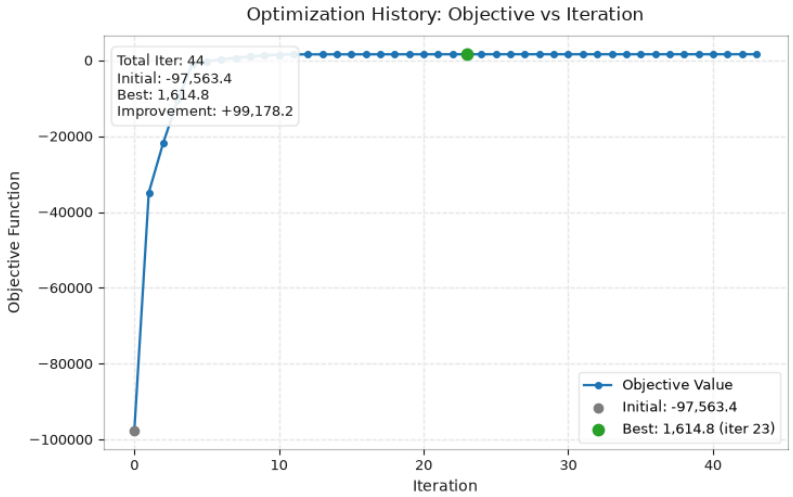
3.2.1.3. Run Ketiga

Initial :



Final :



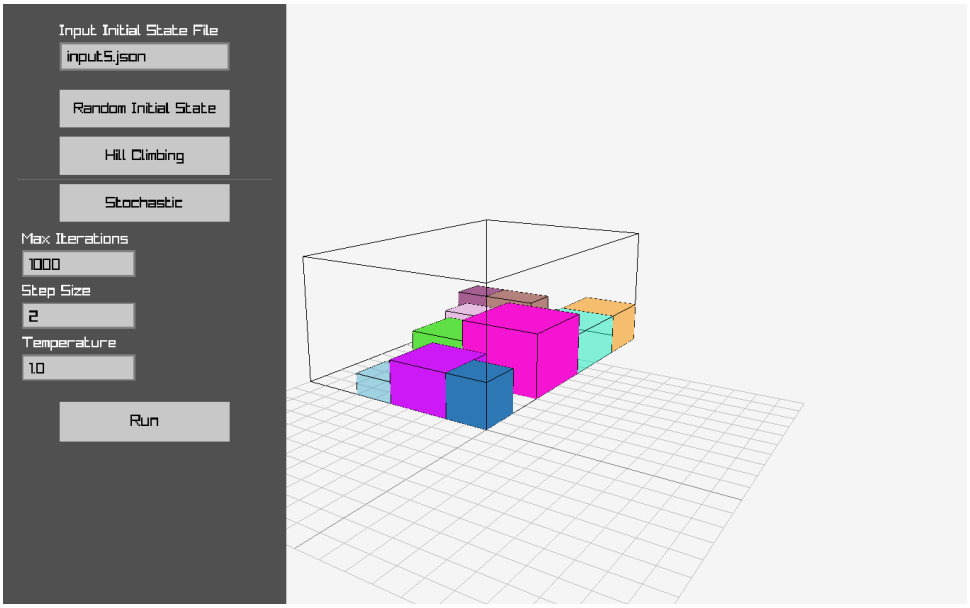


3.2.2. Analisis

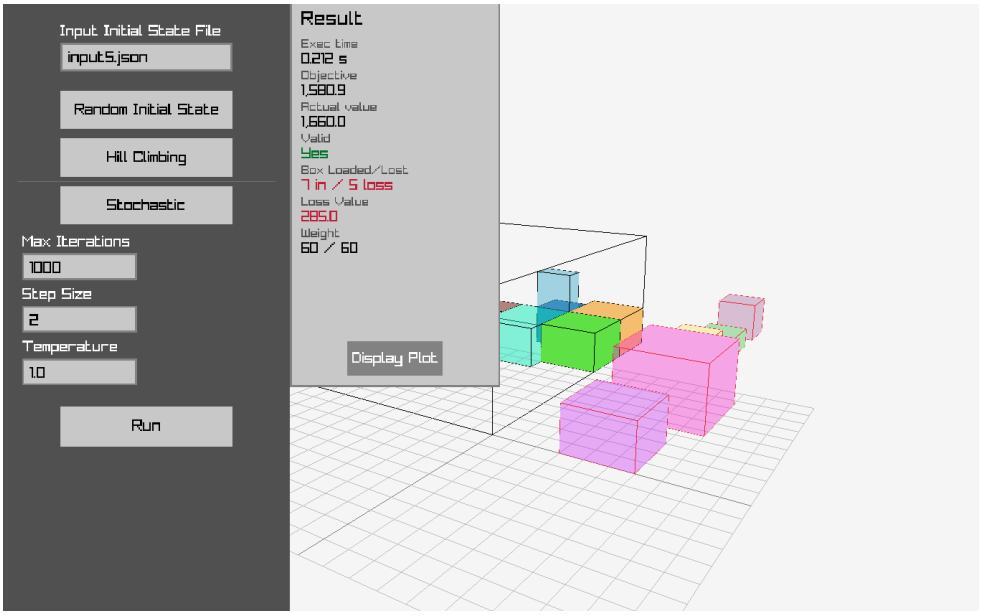
3.3. Stochastic Hill Climbing

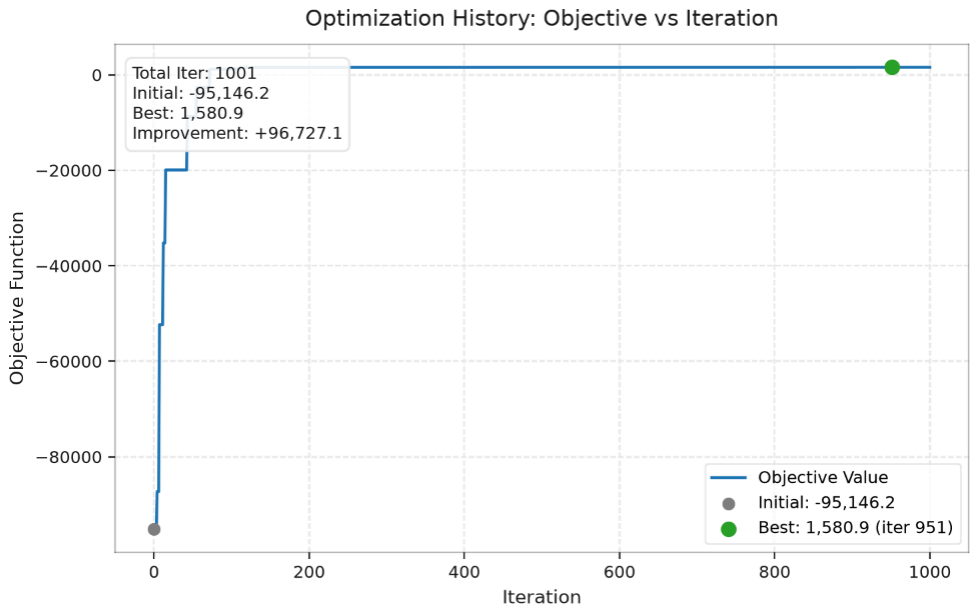
3.3.1. Hasil

Initial:


Input Initial State File
input5.json
Random Initial State
Hill Climbing
Stochastic
Max Iterations
1000
Step Size
2
Temperature
1.0
Run

Final:


Input Initial State File
input5.json
Random Initial State
Hill Climbing
Stochastic
Max Iterations
1000
Step Size
2
Temperature
1.0
Run
Result
Exec time
0.212 s
Objective
1,580.9
Actual value
1,660.0
Valid
Yes
Box Loaded/Lost
7 in / 5 loss
Loss Value
285.0
Weight
60 / 60
Display Plot



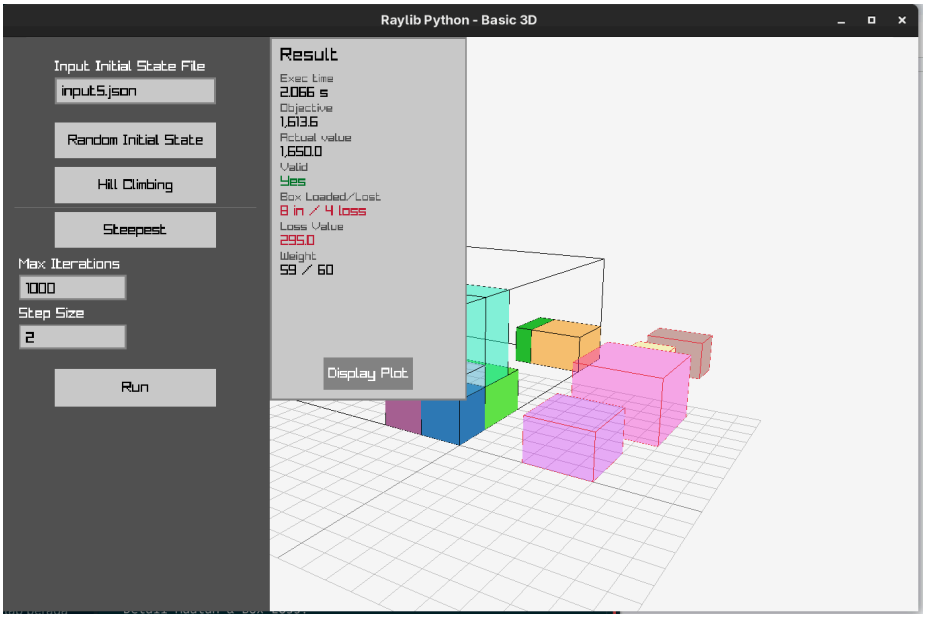
3.3.2. Analisis

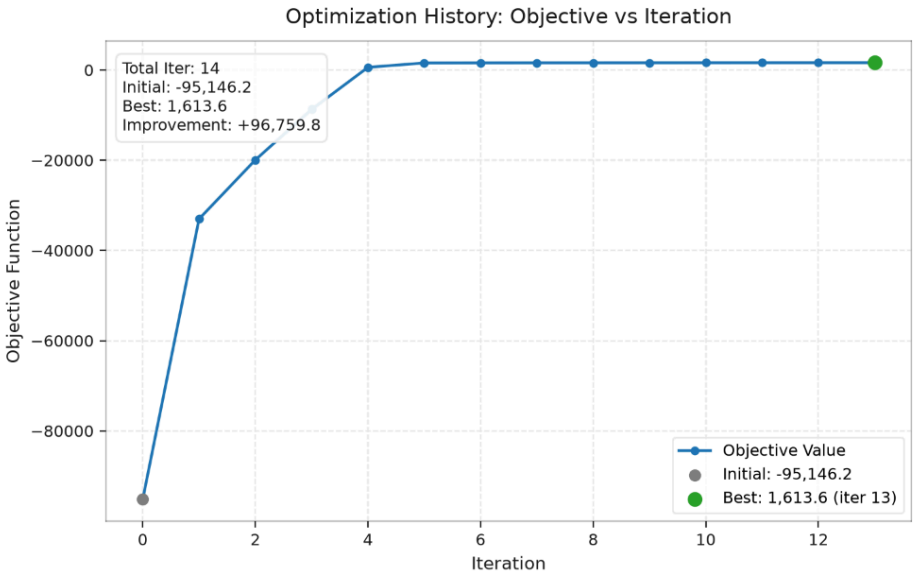
3.4. Random Restart Hill Climbing

3.4.1. Hasil

Initial:

Hasil:



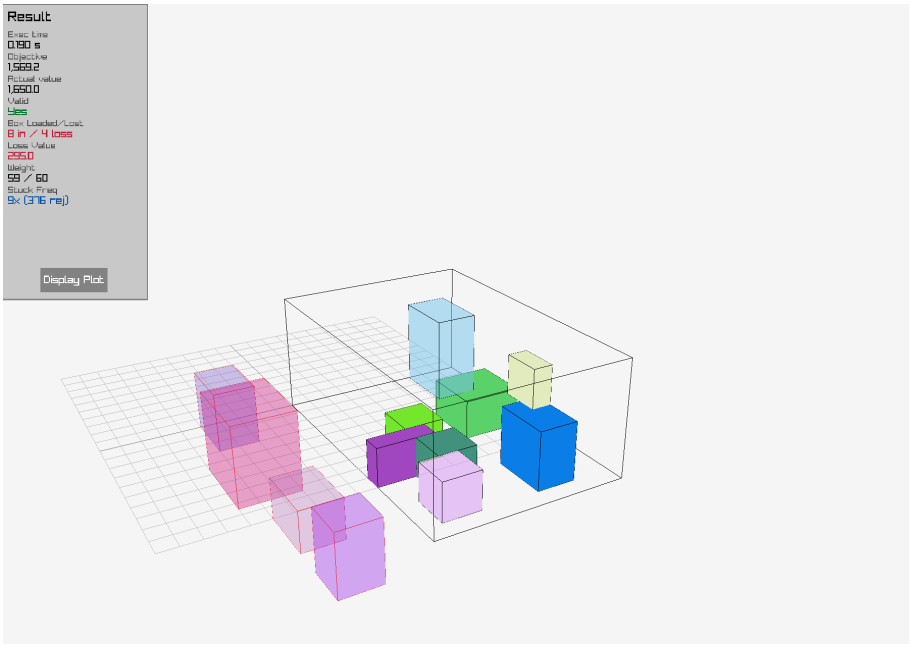


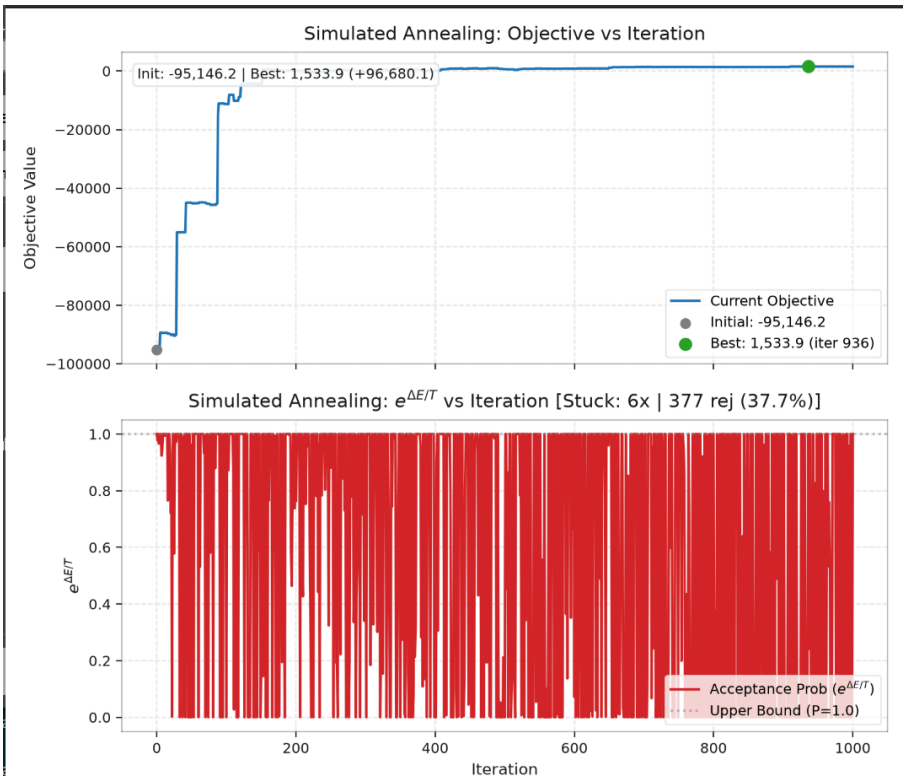
3.4.2. Analisis

3.5. Simulated Annealing

3.5.1. Hasil

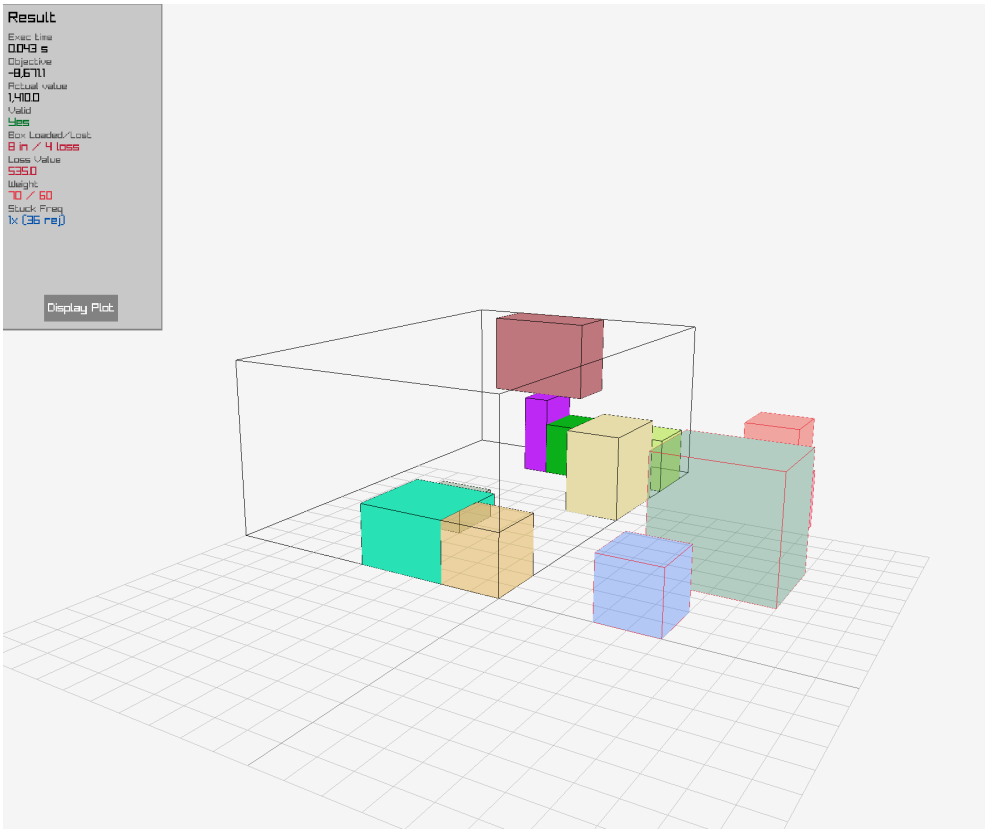
Run Pertama dengan Initial Temp 1000 cooling rate 0.995 min temp 0.001 max iteration 1000 steps per temp 1 dan step size 2

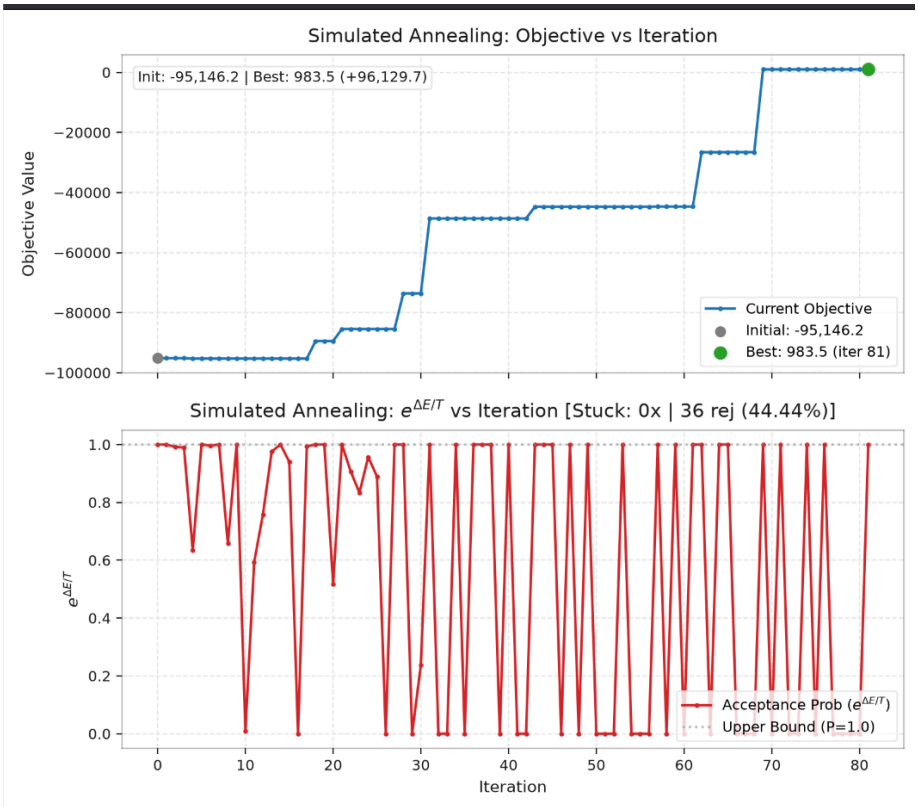




3.5.2. Analisis

3.5.3. Run Kedua dengan Initial Temp 500 cooling rate 0.85 min temp 0.001 max iteration 1000 steps per temp 1 dan step size 2



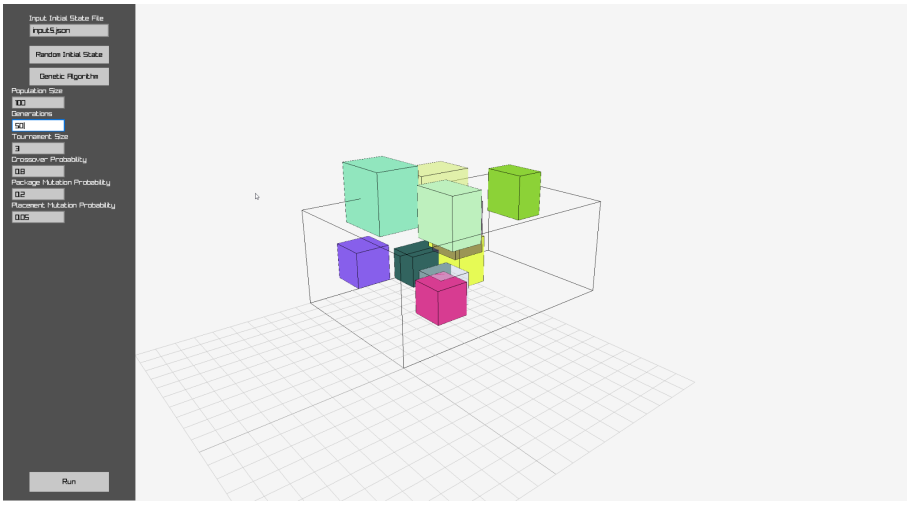


3.6. Genetic Algorithm

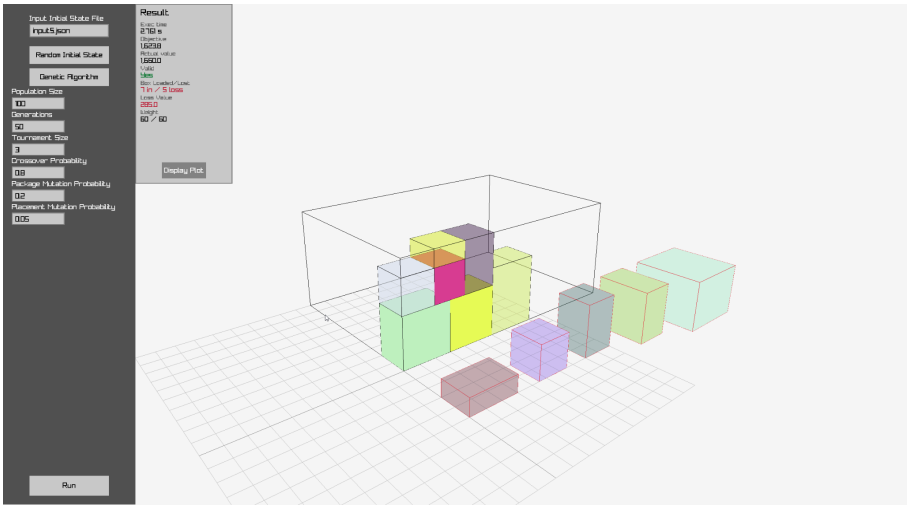
3.6.1. Hasil

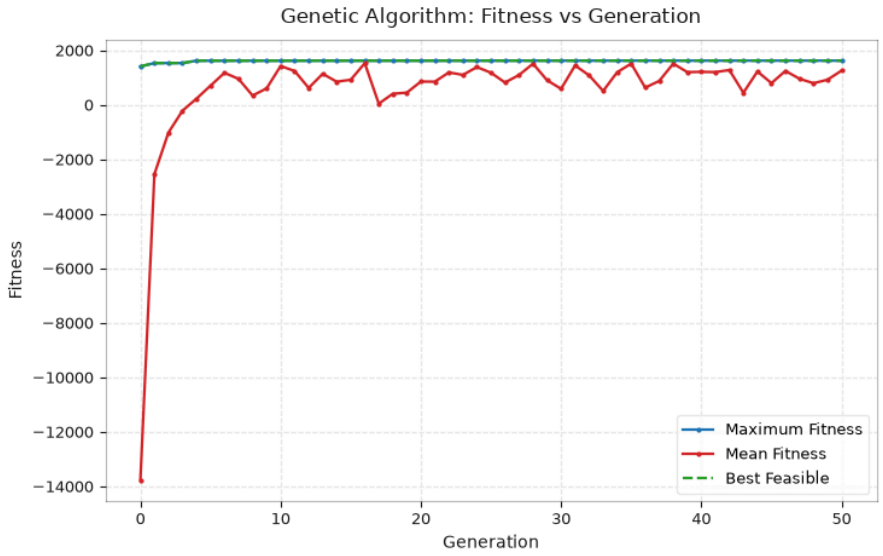
3.6.1.1. Run Pertama dengan Populasi=100 dan Generations=50

Initial :



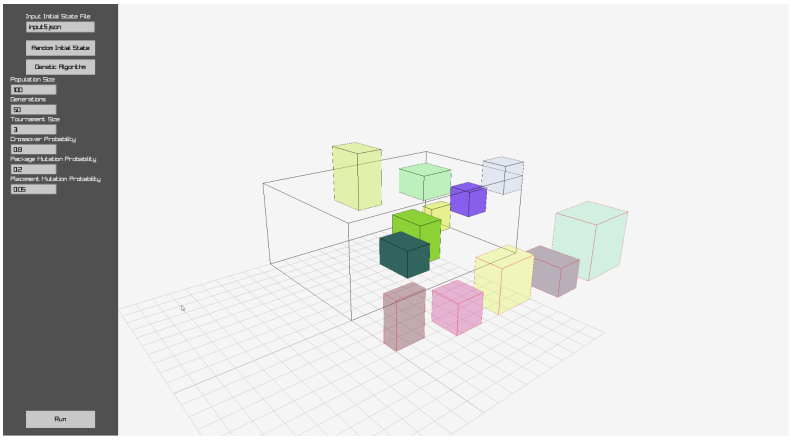
Final :



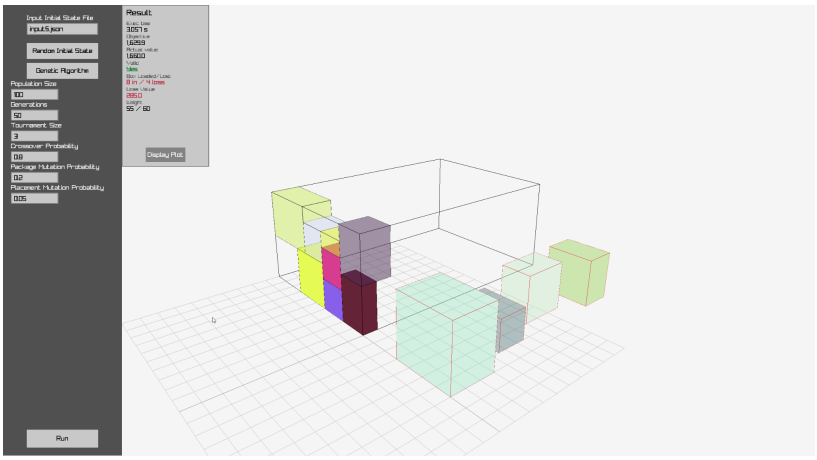


3.6.1.2. Run Kedua dengan Populasi=100 dan Generations=50

Initial :

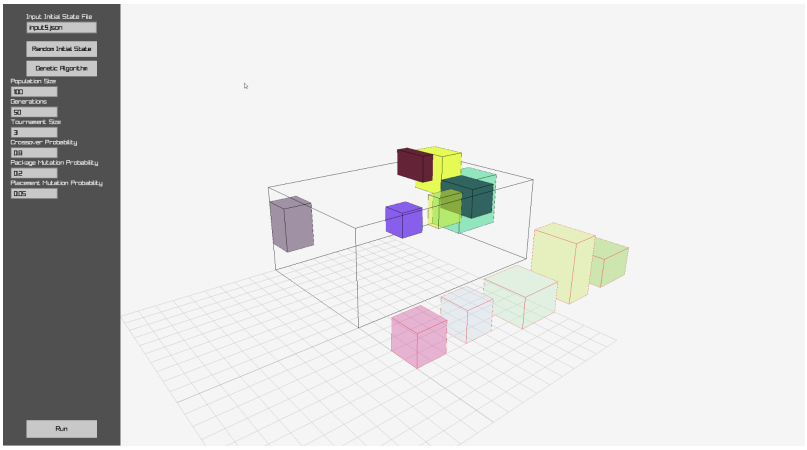


Final :

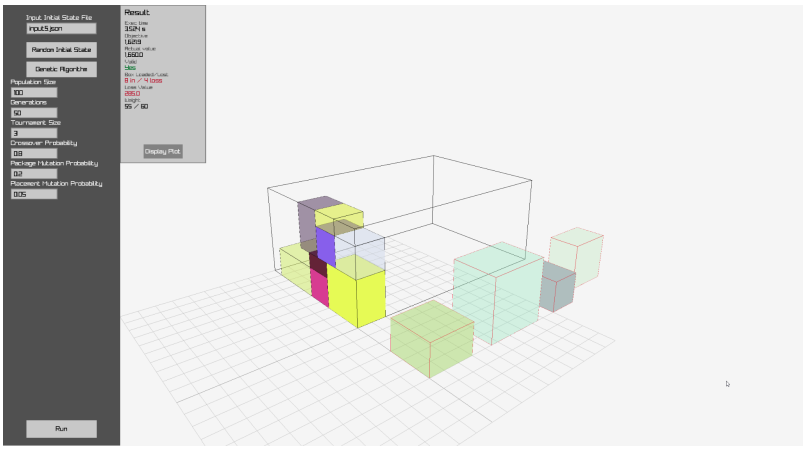


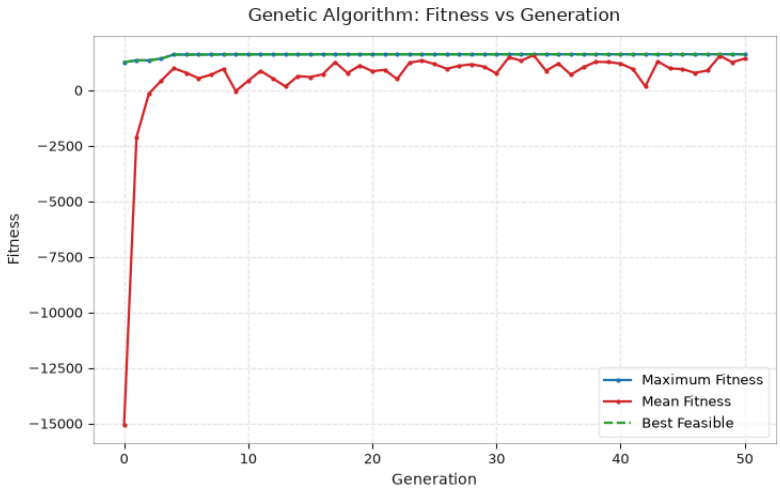
3.6.1.3. Run Ketiga dengan Populasi=100 dan Generations=50

Initial :



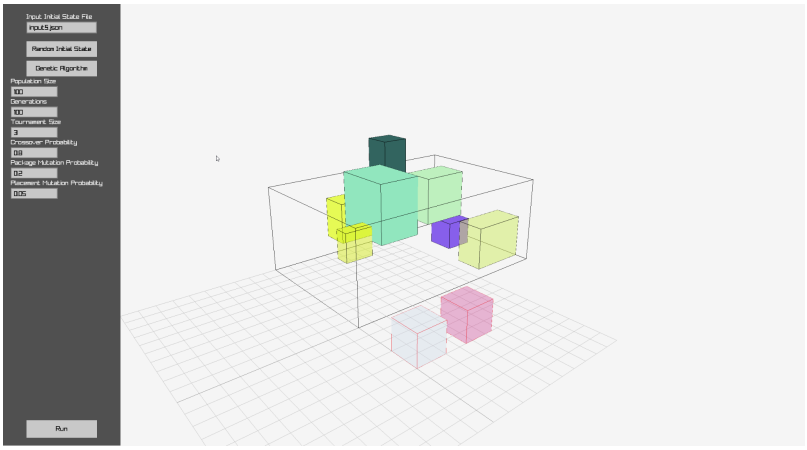
Final :



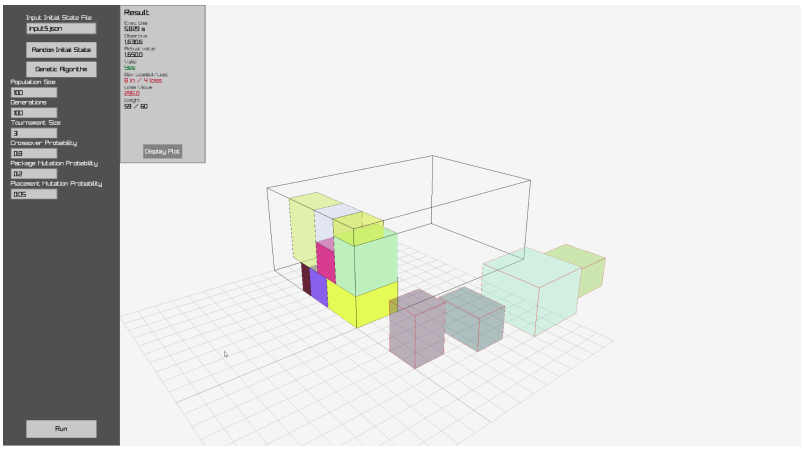


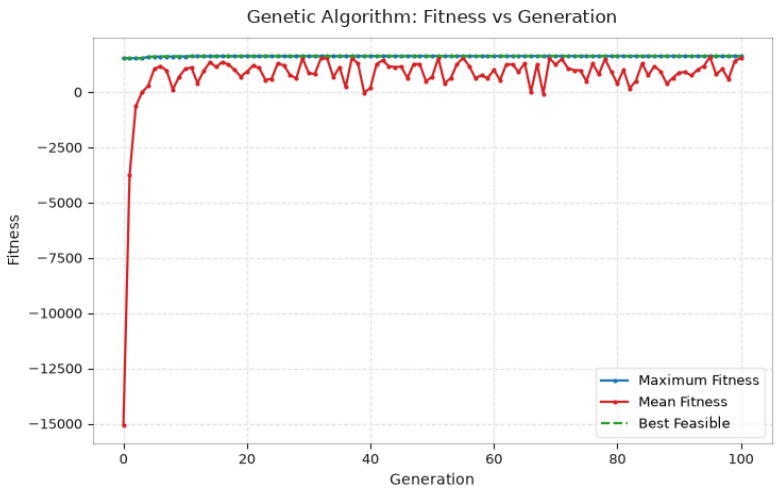
3.6.1.4. Run Pertama dengan Populasi=100 dan Generations=100

Initial :



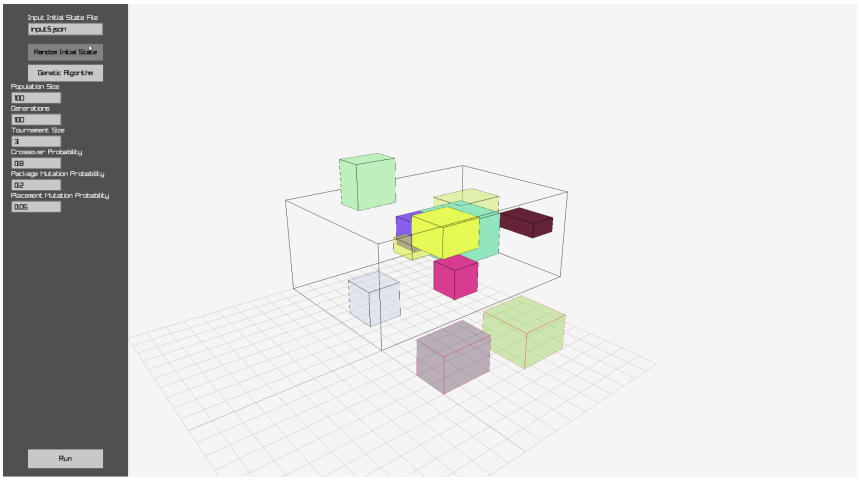
Final :



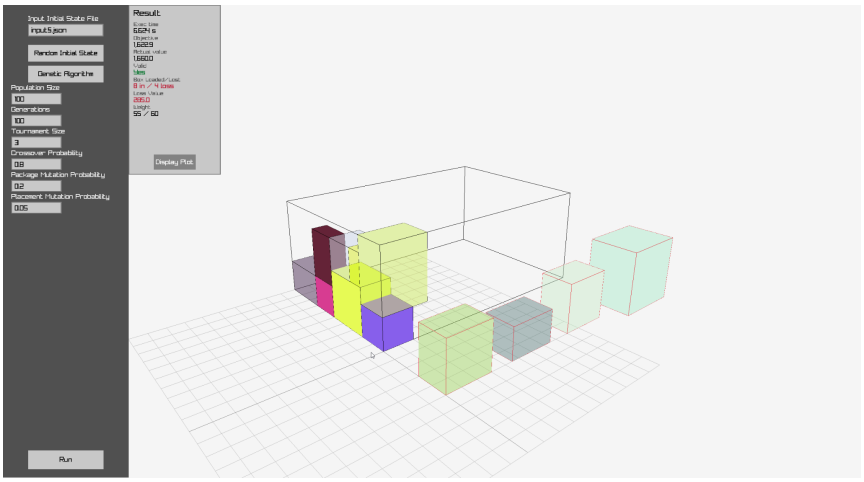


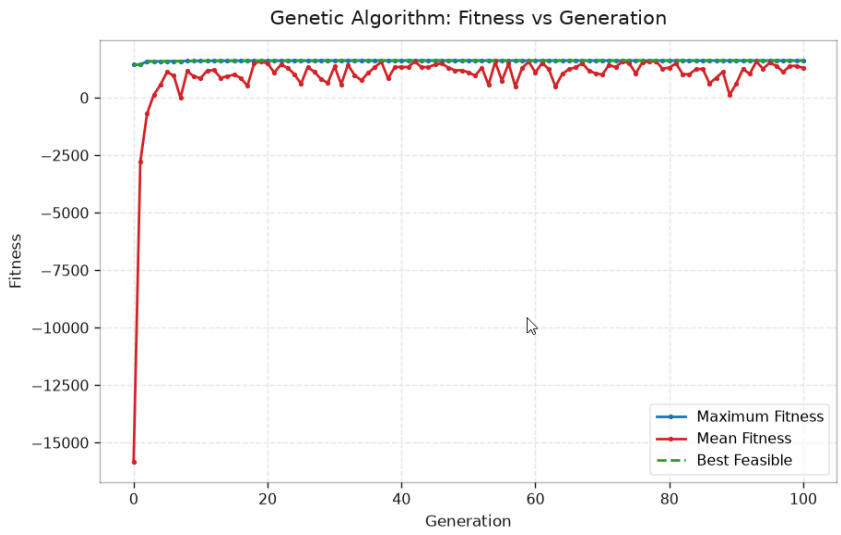
3.6.1.5. Run Kedua dengan Populasi=100 dan Generations=100

Initial :



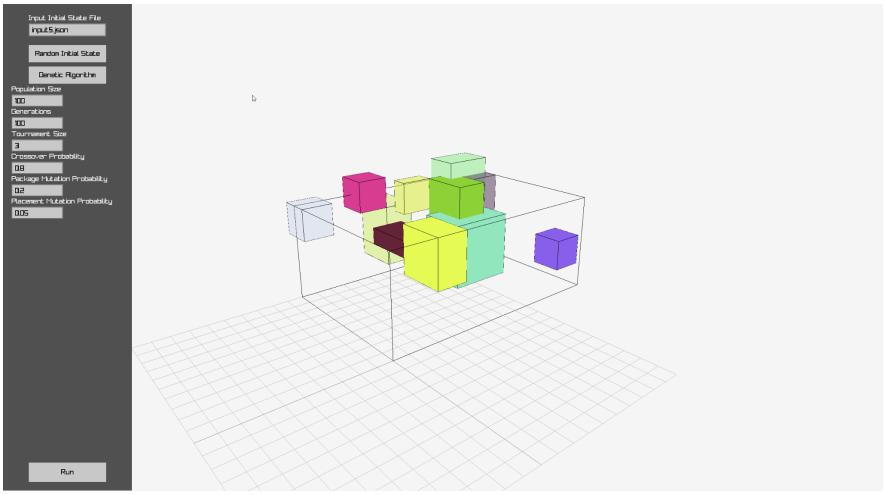
Final :



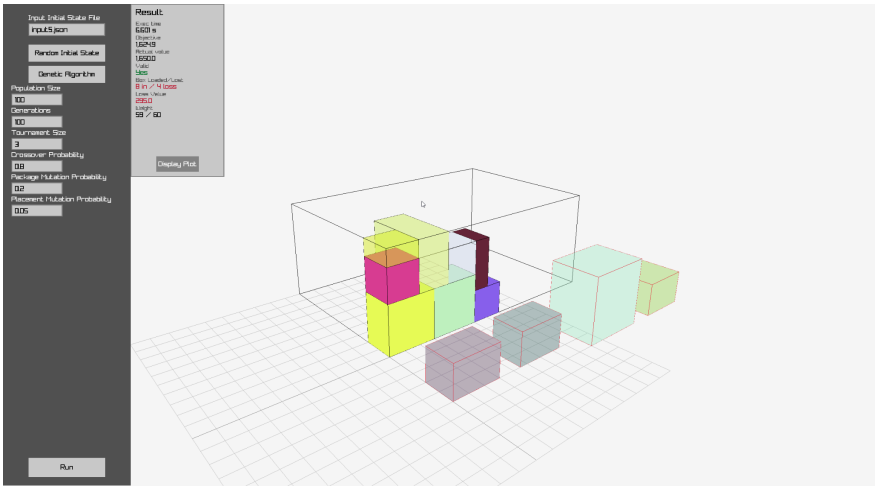


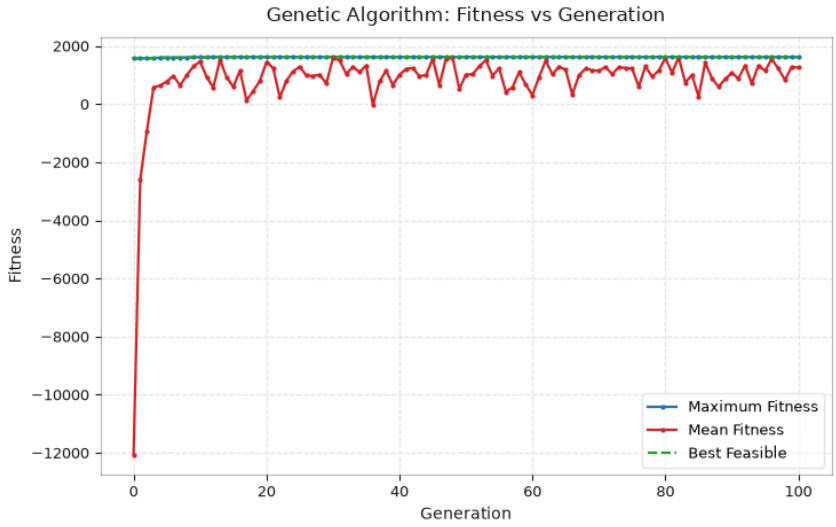
3.6.1.6. Run Ketiga dengan Populasi=100 dan Generations=100

Initial :



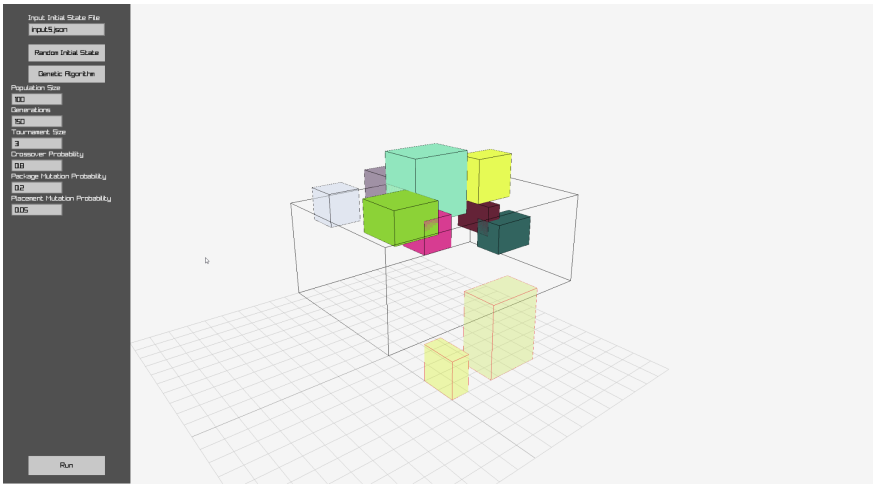
Final :



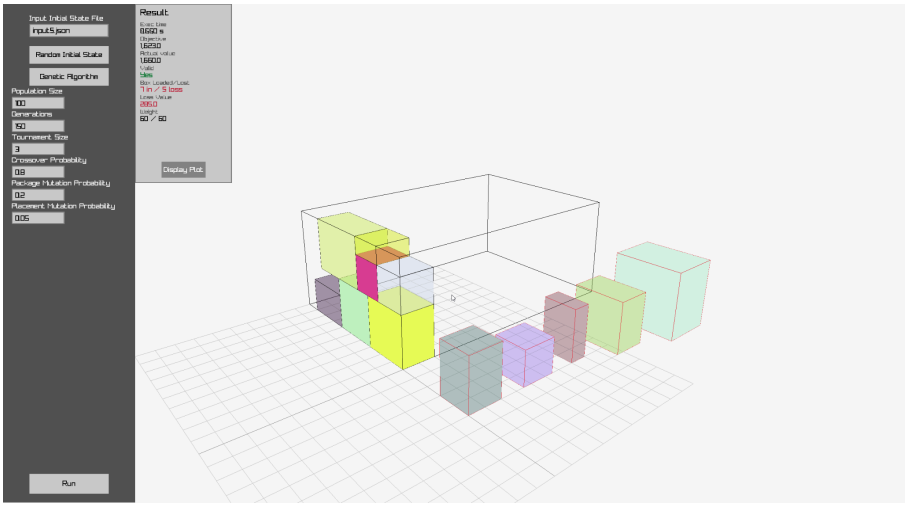


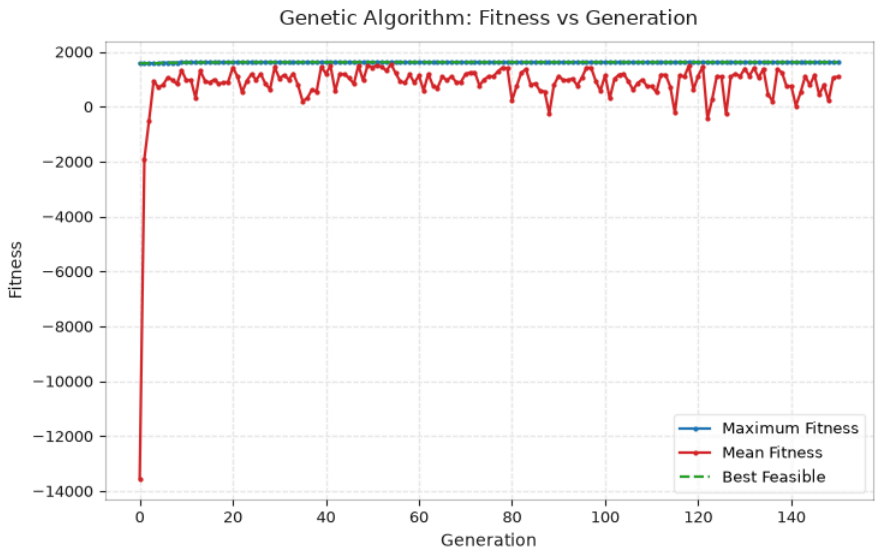
3.6.1.7. Run Pertama dengan Populasi=100 dan Generations=150

Initial :


Input Initial State File
input5.json
Random Initial State
Genetic Algorithm
Population Size
100
Generations
150
Tournament Size
3
Crossover Probability
0.8
Package Mutation Probability
0.2
Placement Mutation Probability
0.05
Run

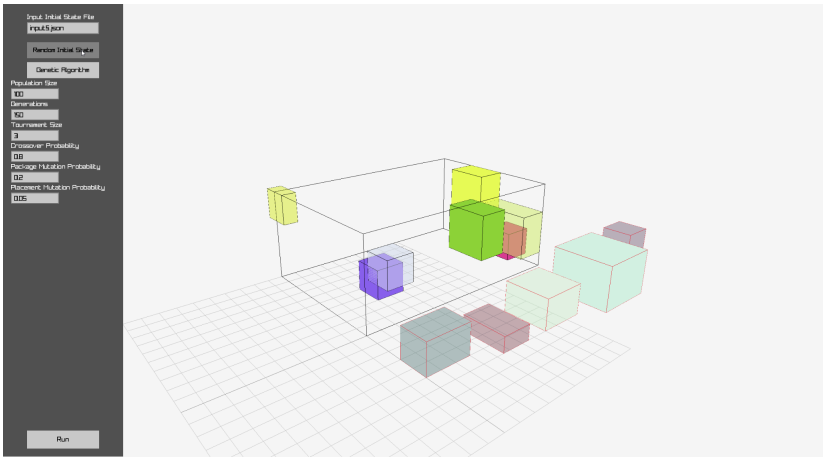
Final :


Input Initial State File
input5.json
Random Initial State
Genetic Algorithm
Population Size
10
Generations
150
Tournament Size
3
Crossover Probability
0.8
Package Mutation Probability
0.2
Placement Mutation Probability
0.05
Run
Result:
Display Plot

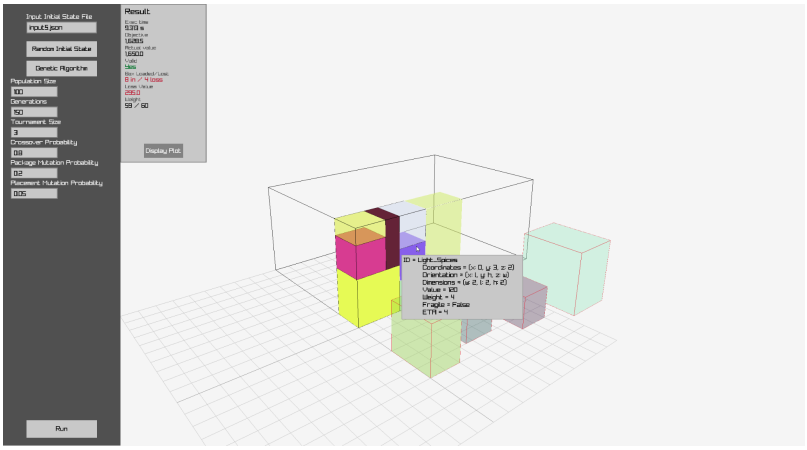


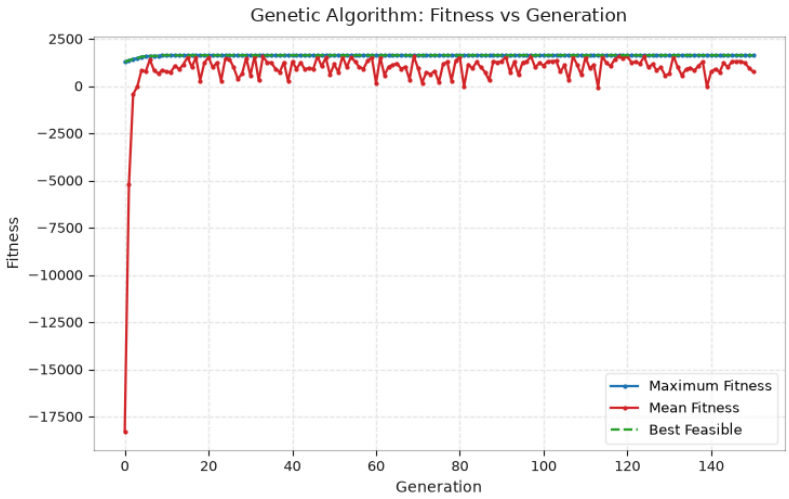
3.6.1.8. Run Kedua dengan Populasi=100 dan Generations=150

Initial :



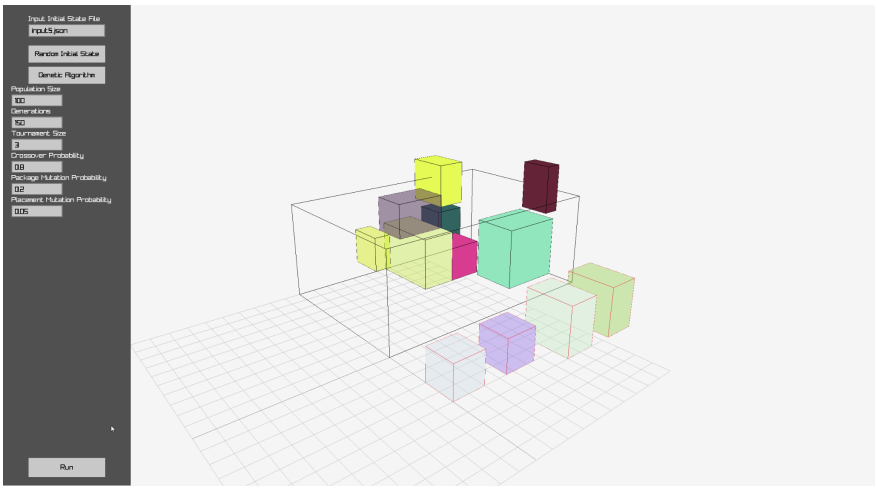
Final :



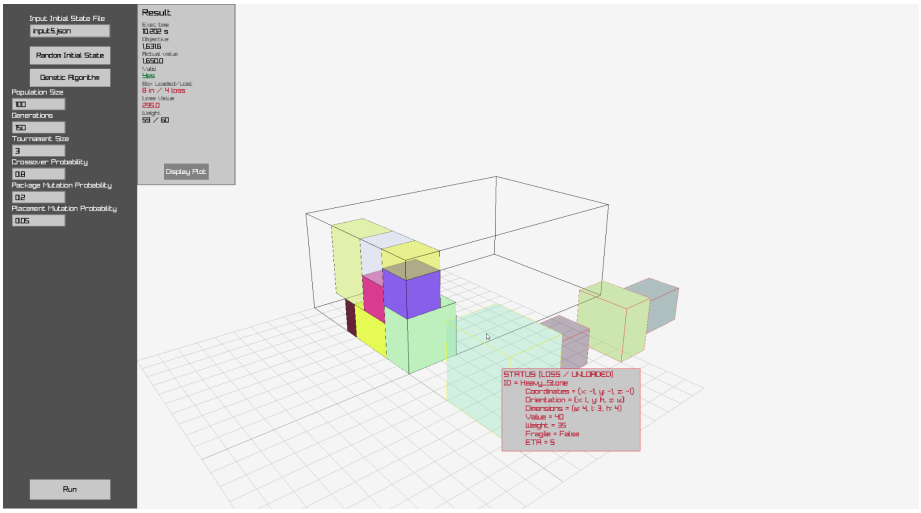


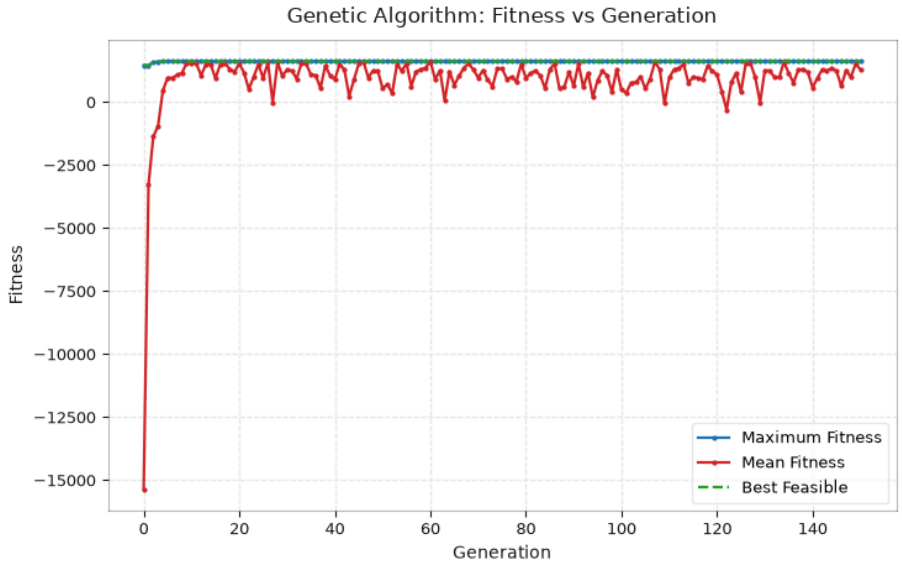
3.6.1.9. Run Ketiga dengan Populasi=100 dan Generations=150

Initial :



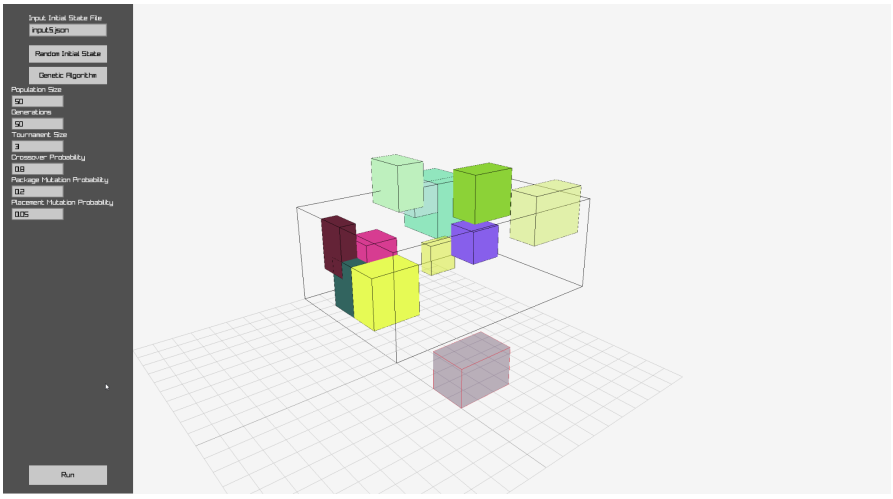
Final :



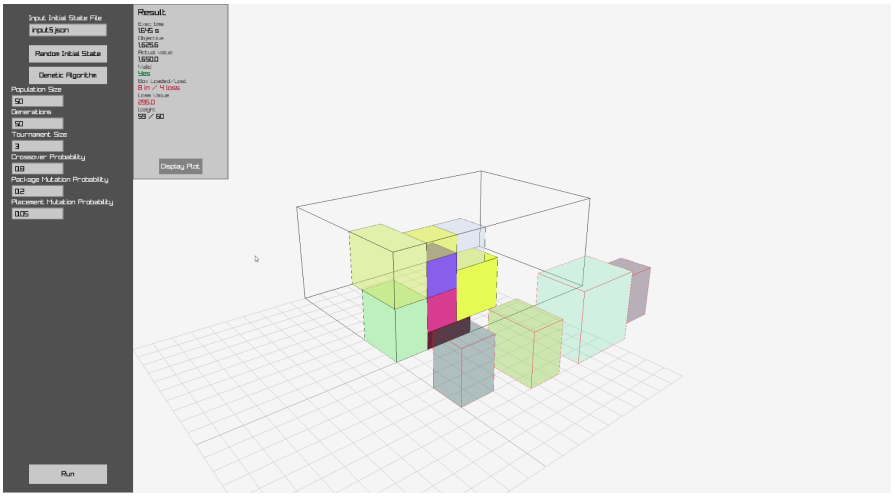


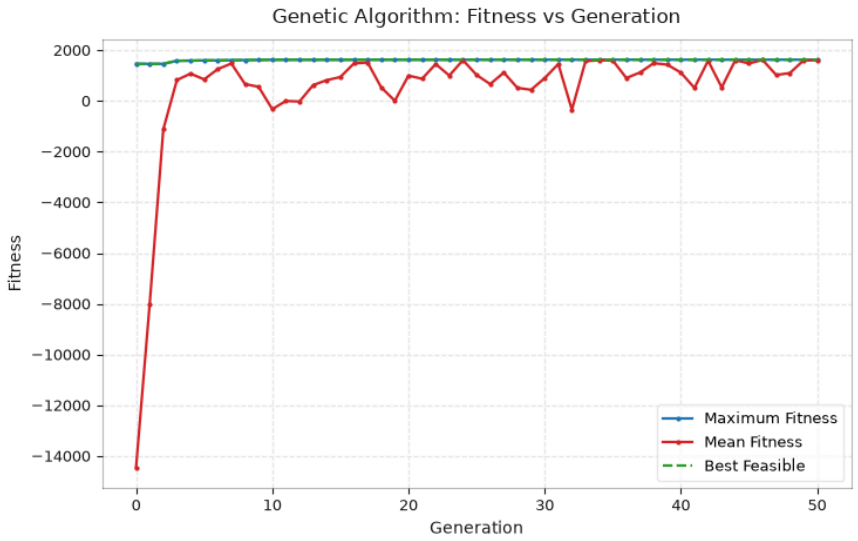
3.6.1.10. Run Pertama dengan Populasi=50 dan Generations=50

Initial :


Input Initial State File
input5.json
Random Initial State
Genetic Algorithm
Population Size
50
Generations
50
Tournament Size
3
Crossover Probability
0.8
Package Mutation Probability
0.2
Placement Mutation Probability
0.05
Run

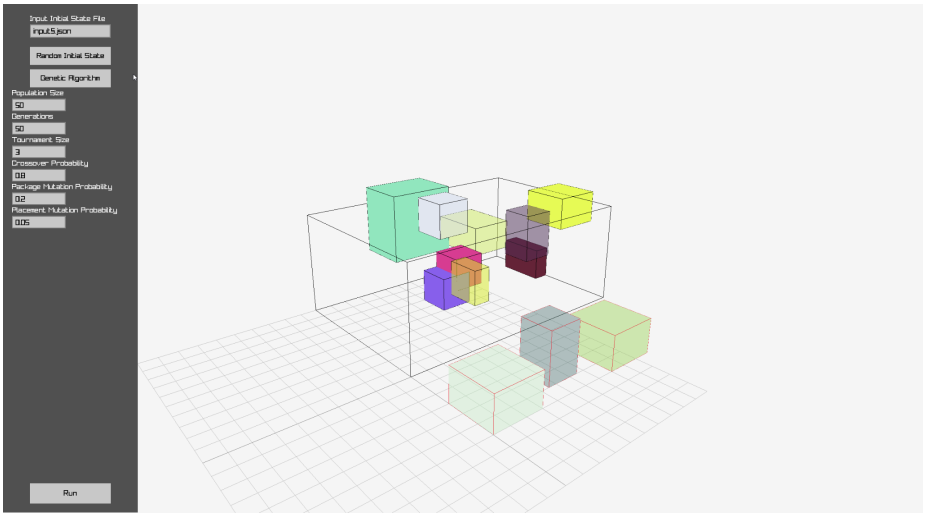
Final :


Input Initial State File
input5.json
Random Initial State
Genetic Algorithm
Population Size
50
Generations
50
Tournament Size
3
Crossover Probability
0.8
Package Mutation Probability
0.2
Placement Mutation Probability
0.05
Run
Result
Display Plot

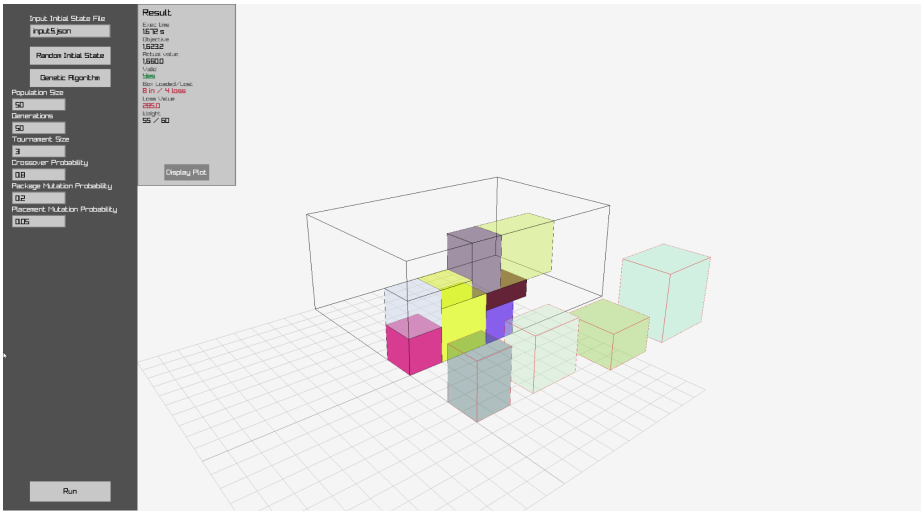

Genetic Algorithm: Fitness vs Generation
Fitness
Generation
2000
0
−2000
−4000
−6000
−8000
−10000
−12000
−14000
0
10
20
30
40
50
Maximum Fitness
Mean Fitness
Best Feasible

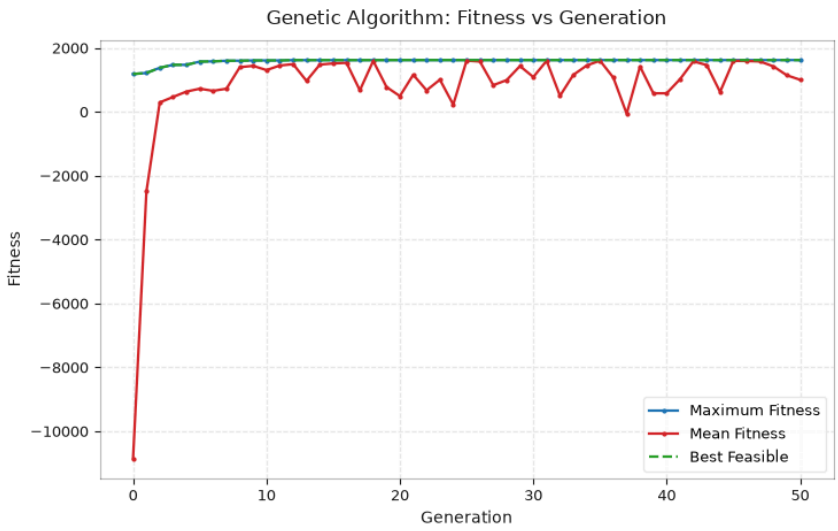
3.6.1.11. Run Kedua dengan Populasi=100 dan Generations=50

Initial :



Final :





3.6.1.12.

Kesimpulan dan Saran

Pembagian Tugas

Tugas	NIM
Visualizer	13524057
Objective Function & Heuristic	13524057
Steepest Ascent Hill Climbing	13524057
Sideways Move Hill Climbing	13524060
Random Restart Hill Climbing	13524130
Stochastic Hill Climbing	13514130
Simulated Annealing	13524130
Genetic Algorithm	13524060

Referensi

[1] Institut Teknologi Bandung, “Beyond Classical Search,” *IF3170 Inteligensi Artifisial*, materi perkuliahan, Semester I Tahun Akademik 2026/2027.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.

[3] T. G. Crainic, G. Perboli, and R. Tadei, “Extreme point-based heuristics for three-dimensional bin packing,” *INFORMS Journal on Computing*, vol. 20, no. 3, pp. 368–384, 2008, doi: 10.1287/ijoc.1070.0250.

[4] J. F. Gonçalves and M. G. C. Resende, “A biased random key genetic algorithm for 2D and 3D bin packing problems,” *International Journal of Production Economics*, vol. 145, no. 2, pp. 500–510, 2013, doi: 10.1016/j.ijpe.2013.04.019.

Lampiran

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM:

1. Benedict
2. Steven Tan (13524060)
3. Faris Wirakusuma (13524130)

Fakultas/Sekolah : STEI - K
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 IF3170 Inteligensi Artifisial

Menyatakan bahwa kami menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

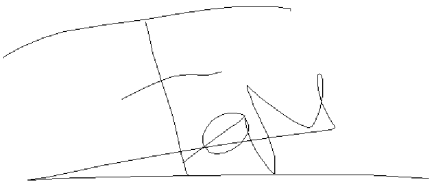
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan

Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	ChatGPT digunakan dalam diskusi ide desain Genetic Algorithm
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Faris Wiraksuma

NIM 13524130

Bandung, 20 Oktober 2026



Steven Tan

13524060

(tempat), (tanggal)

(tanda tangan)

(Nama Lengkap)

NIM